

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования «Казанский национальный исследовательский
технический университет им. А.Н. Туполева-КАИ»
(КНИТУ-КАИ)

Институт компьютерных технологий и защиты информации

Кафедра компьютерных систем

Направление подготовки: 09.03.01 «Информатика и вычислительная техника»

Образовательная программа: Вычислительные машины, комплексы системы и сети

К защите допустить

Зав. каф. КС

_____/Вершинин И.С.

«__» мая 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему: «Разработка онлайн-сервиса на основе нейросети для
цветокоррекции графических изображений»

ОБУЧАЮЩИЙСЯ Черногузов Андрей Владимирович _____
(фамилия, имя, отчество) (подпись)

РУКОВОДИТЕЛЬ канд. техн. наук, Минязев Ринат Шавкатович _____
(ученая степень, звание, фамилия, имя, отчество) (подпись)

Казань 2025 г.

Development of an online neural network-based service for color correction of graphic images

By
Andrew Vladimirovich Chernoguzov

Submitted to the Department of Computer Systems

in partial fulfillment of the Requirements for the degree of

BACHELOR OF SCIENCE

at the

Federal State Budgetary Educational Institution of Higher Education
«Kazan National Research Technical University named after A.N.Tupolev-KAI»
(KNRTU-KAI)

Author

Andrey Vladimirovich
Chernoguzov

(signature)

Supervisor

Rinat Shavkatovich Minyazev

PhD, Department of Computer
Systems

(signature)

Certified by

Igor Sergeevich Vershinin

PhD, Head of the Department of
Computer Systems

(signature)

date

3 Исходные данные к выпускной квалификационной работе

4 Перечень подлежащих разработке вопросов и исходные данные к ним:

4.1 _____

5 Перечень графического материала (при наличии):

Презентация на слайдах _____

6 Консультанты по ВКР (при их наличии, с указанием относящихся к ним разделов):

(наименование раздела, ФИО консультанта, подпись)

(наименование раздела, ФИО консультанта, подпись)

(наименование раздела, ФИО консультанта, подпись)

(наименование раздела, ФИО консультанта, подпись)

Дата выдачи задания « 03 » февраля 2025г.

Руководитель ВКР	_____	_____
	(подпись)	(ФИО)
Задание к исполнению принял	_____	_____
	(подпись)	(ФИО)

Календарный план выполнения ВКР

№ п/п	Наименование этапов (разделов) выпускной квалификационной работы	Срок выполнения этапов (разделов) ВКР	Примечание
1	Получение задания ВКР.	03.02.25	
2	Изучение доступных научных материалов о метод цветокоррекции изображений.	10.02.25	
3	Исследование аналогов, изучение их принципа ра- боты.	15.02.25	
4	Анализ технологий для разработки программного обеспечения.	21.02.25	
5	Изучение программных средств (Среда разработки Visual Studio Code)	28.02.25	
6	Изучение программных средств (Библиотеки Py- thon: Pillow, Aiogram)	15.03.25	
7	Разработка базовых модулей для Telegram бота	30.03.25	
8	Разработка модуля цветокоррекции изображения	15.03.25	
9	Тестирование разработанных модулей и устране- ние ошибок при проектировании	10.04.25	
10	Тестирование сервиса пользователями, сбор ре- зультатов тестирования и устранение ошибок.	05.05.25	
11	Формирование пояснительной записки и презента- ции к ВКР	10.05.25	

Обучающийся _____
(подпись)
(Фамилия, инициалы)
(дата)

Руководитель _____
(подпись)
(Фамилия, инициалы)
(дата)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	8
ГЛАВА 1. АНАЛИТИЧЕСКИЙ ОБЗОР	9
1.1 Введение в проблематику цветокоррекции	9
1.2 Обзор существующих аналогов	11
1.2.1 Let's Enhance	12
1.2.2 Adobe Firefly	14
1.2.3 Fotor (ИИ режимы)	16
1.3 Сравнительный анализ аналогов	18
1.4 Выводы по разделу	19
ГЛАВА 2. ТЕОРИТИЧЕСКИЕ ВОПРОСЫ ПРОЕКТИРОВАНИЯ	20
2.1 Постановка задачи	20
2.2 Общая архитектура решения	22
2.2.1 Пользовательский интерфейс	25
2.3 Используемые технологии и библиотеки	29
2.3.1 Python	30
2.3.2 Aiogram	32
2.3.3 Pillow (PIL-Fork)	34
2.3.4 NumPy	36
2.3.5 Replicate API и Stable Diffusion (SDXL)	37
2.3.6 Aiogram-FSM	39
2.4. Обработка изображений: методы и алгоритмы	40
2.4.1 Автоматическая цветокоррекция	41
2.4.2 Пресеты цветовых стилей	42
2.4.3 Коллаж «до/после»	43
2.4.4 Нейросетевая стилизация (AI Style)	44
2.5.1 Реализация процесса в Telegram-боте	47
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	48
3.1 Структура проекта	48
3.2 Интеграция нейросети	51
3.3 Интерфейс пользователя, обработка ошибок и безопасность	54
3.4 Тестирование и отладка	58
3.5 Развёртывание и запуск системы	59
ЗАКЛЮЧЕНИЕ	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ	62
ПРИЛОЖЕНИЕ 1 ЛИСТИНГ ФАЙЛА УПРАВЛЕНИЯ КОРРЕКЦИЕЙ	63
ПРИЛОЖЕНИЕ 2 ЛИСТИНГ ФАЙЛА КОНФИГУРАЦИИ СЕРВИСА	70

ПРИЛОЖЕНИЕ 3 ЛИСТИНГ ОСНОВНОГО ФАЙЛА СЕРВИСА	71
ПРИЛОЖЕНИЕ 4 ДИАГРАММА СЦЕНАРИЕВ ВЗАИМОДЕЙСТВИЯ С БОТОМ.....	72

АННОТАЦИЯ

Исследуются сервисы для цветокоррекции изображений. Предлагается онлайн-сервис на основе нейросетевых технологий. Представлен анализ существующих решений, определены ключевые преимущества применения глубокого обучения в области обработки изображений. Демонстрируется, что использование нейросетей позволяет добиться повышения точности цветовой передачи на 20–35% по сравнению с традиционными методами. Обсуждаются особенности реализации внутренних и внешних частей (backend- и frontend-частей) сервиса, а также перспективы интеграции в профессиональные рабочие процессы дизайнеров и фотографов.

Ключевые слова: цветокоррекция, нейросети, глубокое обучение, онлайн-сервис, Python, OpenCV, генеративно-состязательные сети, цифровая обработка изображений, облачные технологии.

Количество страниц: 52

Количество иллюстраций: 14

Количество приложений: 4

Количество использованных источников: 16

ABSTRACT

Services for color correction of images are being investigated. An online service based on neural network technologies is offered. An analysis of existing solutions is presented, and the key advantages of using deep learning in the field of image processing are identified. It is demonstrated that the use of neural networks makes it possible to achieve an increase in the accuracy of color transmission by 20-35% compared with traditional methods. The specifics of the implementation of the backend and frontend parts of the service are discussed, as well as the prospects for integration into the professional work processes of designers and photographers.

Keywords: color correction, neural networks, online service, Python, OpenCV, GAN, digital image processing, cloud technologies.

Page count: 52

Picture count: 14

Appendix count: 4

Reference count: 15

ВВЕДЕНИЕ

Визуальный контент играет ключевую роль в цифровой среде – от рекламы и социальных сетей до электронной коммерции и дизайна. Качественная обработка изображений, в частности цветокоррекция, существенно влияет на восприятие, вовлечённость аудитории и даже на поведенческие факторы потребителей. Корректно подобранные цветовые решения усиливают эмоциональное воздействие, делают изображение более выразительным и профессиональным. Однако традиционная цветокоррекция требует серьёзных знаний, навыков работы с графическими редакторами и значительных временных затрат, что делает её недоступной для большинства пользователей без соответствующей подготовки.

Современные технологии машинного обучения и компьютерного зрения открывают новые возможности для автоматизации этой задачи. Модели искусственного интеллекта, способные анализировать содержимое изображения и адаптировать цветовые параметры с учётом контекста, значительно сокращают трудозатраты и повышают точность коррекции. В последние годы активно развиваются решения, позволяющие выполнять базовую обработку изображений онлайн, но большинство из них либо ограничены в функциональности, либо ориентированы на узкие задачи и не предоставляют гибких настроек.

Актуальность данной работы обусловлена потребностью в универсальном и доступном инструменте, сочетающем в себе удобство использования, высокое качество цветокоррекции и возможности адаптации под различные типы изображений. Целью выпускной квалификационной работы является разработка онлайн-сервиса, использующего технологии искусственного интеллекта для автоматической цветокоррекции графических изображений. Такой сервис должен обеспечивать быструю, точную и контекстно-зависимую обработку визуального контента, доступную как для профессионалов, так и для широкой аудитории.

Работа включает в себя аналитический обзор существующих решений, выбор и обучение архитектуры искусственного интеллекта, разработку пользовательского интерфейса и тестирование системы на различных наборах изображений. В результате планируется создать прототип веб-приложения, демонстрирующий преимущества автоматизированной цветокоррекции с применением современных технологий искусственного интеллекта.

ГЛАВА 1. АНАЛИТИЧЕСКИЙ ОБЗОР

1.1 Введение в проблематику цветокоррекции

Обработка графических изображений в современном мире является важной частью множества процессов, например, профессиональная фотография, маркетинг и оформление социальных сетей. Одним из важных этапов обработки изображения является – цветокоррекция. Цветокоррекция – это изменение цветов и тонов на цифровом изображении чтобы достичь определённого визуального эффекта, коммуникативных целей или убрать дефекты. Цветокоррекция непременно влияет на конверсию в электронной коммерции, которая увеличивается на 10-15%, и на вовлеченность в социальных сетях, которая получается на 30% больше вовлечения с изображениями с гармоничной палитрой [1].

В современном мире множество инструментов, которые требуют навыков их использования и не доступны обычному пользователю. Самые популярные графические редакторы на сегодняшний день – Adobe Photoshop и Lightroom. Каждый день этими приложениями пользуются профессиональные графические дизайнеры, фотографы и ретушеры, которые тратят очень много времени на то, чтобы, используя внутренние инструменты, реализовать их задумку. В то время как для новичков эта задача становится очень сложной, потому что требует экспертные знания, без которых результат может приводить к перенасыщенным тонам кожи в портретах или искажению текстур в пейзажах. Стоит отметить, что на данный момент существуют автоматизированные системы цветокоррекции, которые обладают ограниченным функционалом, либо не обеспечивают достаточного качества обработки. Многие алгоритмы, основанные на простых правилах (например, нормализация гистограмм), не учитывают семантику изображения: коррекция, идеальная для пейзажа, может испортить портрет, а готовые настройки, которые называют в профессиональном сообществе – “пресеты”, не адаптируются к изменению освещения или особенностям цветопередачи камеры. Кроме того, онлайн-решения часто жертвуют точностью в пользу скорости, используя упрощенные модели, что приводит к артефактам или потере детализации [1, 2].

Такого рода проблемы подчеркивают потребность в более простых, эффективных и доступных решениях, способных обеспечивать скорость, точность и правильность обработки близкой к ручной обработке. Современные технологии в сфере искусственного интеллекта, такие как генеративно-состязательные сети, открывают новые возможности, например, они могут

анализировать изображение контекстно, распознавать объекты и адаптировать обработку под специфику изображения. Это позволяет минимизировать ручное вмешательство, значительно ускоряя процесс [3, 4].

1.2 Обзор существующих аналогов

В настоящее время рынок предлагает несколько категорий решений для автоматической цветокоррекции, обладающих базовыми возможностями для несложной обработки. Однако, для более профессиональной обработки пользователям приходится обращаться к платным сервисам, так как содержание дата-центров, где работают нейронные сети, осуществляется за счет тарифных планов. Далее приведены примеры таких существующих решений.

1.2.1 Let's Enhance

Let's Enhance – это популярная онлайн-платформа, ориентированная на улучшение качества цифровых изображений с использованием технологий искусственного интеллекта. интернет-магазинов, где визуальная составляющая играет ключевую роль. Стартовая страница проекта Let's Enhance (рисунок 1.1). Сервис позиционирует себя как инструмент для автоматической ретуши, позволяющий решать задачи, которые традиционно требуют профессиональных навыков работы в графических редакторах. Основной акцент платформы сделан на повышении разрешения изображений (upscaling) и устранении цифровых шумов, что делает её востребованной среди фотографов, дизайнеров, маркетологов и владельцев.

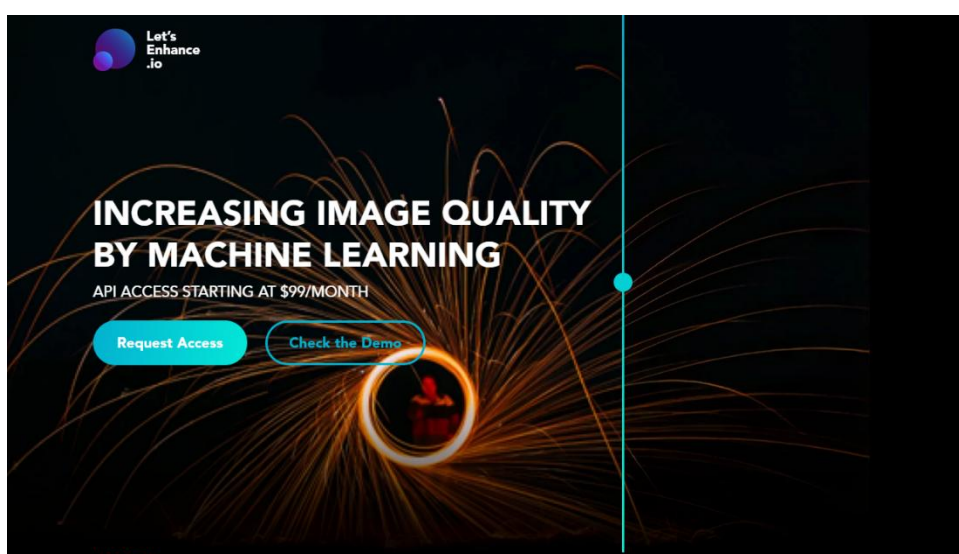


Рисунок 1.1 – Приветственная страница сервиса Let's Enhance

Функционал Let's Enhance включает в себя три возможности, такие как увеличение разрешения и детализации изображений, шумоподавление (устранение дефектов, возникающих в процессе съемки в условиях низкой освещенности) и базовая цветокоррекция, которая реализована через автоматическую настройку баланса белого, изменения контраста и насыщенности. Данный функционал позволяет привести изображение к нормальному виду [1, 5].

В основе технологического стека Let's Enhance лежат генеративно-сопоставительные сети (рисунок 1.2), которые обучены восстанавливать и дополнять детали изображений на основе больших датасетов.



Рисунок 1.2 – Принцип работы генеративно-состязательных сетей

Например, при увеличении размера и разрешения изображения нейросеть начинает предсказывать недостающие пиксели, опираясь на паттерны, характерные для похожих объектов.

К преимуществам данного сервиса относятся высокая скорость обработки, которая важна в современном мире и позволяет пользователю выполнять обработку за 10 – 30 секунд. Кроме того, платформа предлагает API для интеграции её в сторонние приложения [2, 5].

Но, по сколько сервис является достаточно мощным, он имеет ряд ограничений, связанных именно с цветокоррекцией. Пользователи не могут тонко настраивать параметры вручную так как отсутствуют инструменты для работы с кривыми, выборочной коррекции цвета и так далее. Алгоритмы стремятся усреднить результат, что приводит к потере авторского стиля. Кроме того, платформа делает упор на увеличение разрешения изображения, а не на цветокоррекцию, поэтому её цветокоррекция уступает в точности специализированным решениям, таким как Adobe Lightroom. Это создаёт проблему для профессионалов, которым требуется не только улучшение резкости, но и глубокая работа с цветом, учитывающая специфику сюжета.

По итогу, стоит отметить, что Let's Enhance демонстрирует потенциал технологий в сфере искусственного интеллекта, что позволяют оптимизировать рутинные задачи пользователя, но остаётся нишевым решением с фокусом на повышении технического качества, а не художественной выразительности. Его пример подчёркивает необходимость разработки сервисов, сочетающих скорость алгоритмов ИИ с гибкостью ручной настройки, особенно в контексте цветокоррекции.

1.2.2 Adobe Firefly

Adobe Firefly представляет собой облачный сервис на основе генеративного искусственного интеллекта, встроенного в Adobe Photoshop (рисунок 1.3), разработанный компанией Adobe в рамках экосистемы Creative Cloud.

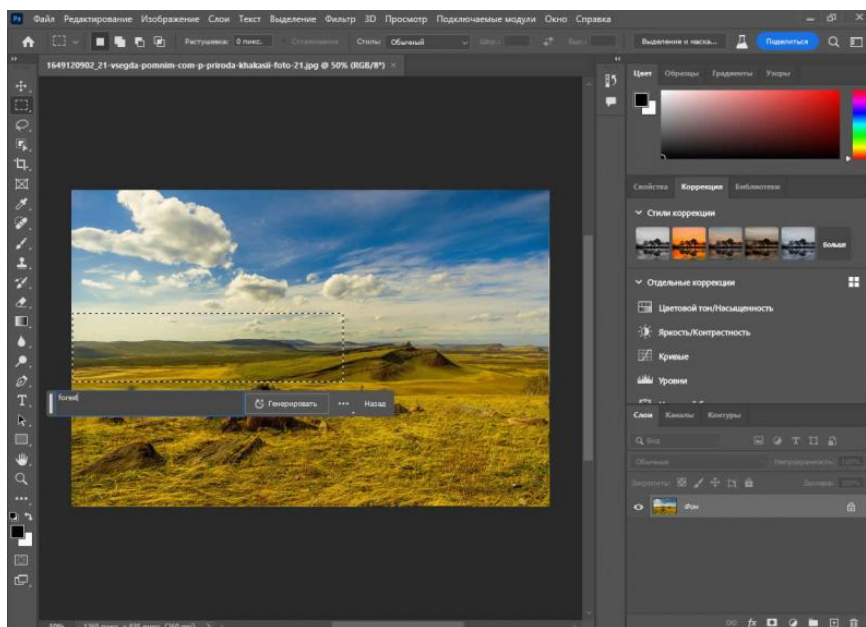


Рисунок 1.3 – Интерфейс Adobe Photoshop со встроенным Adobe Firefly

Firefly позволяет не только редактировать существующую фотографию, но и генерировать совершенно новое изображение, включая векторную графику и текстовые эффекты. Это делает данный продукт универсальным решением для дизайнеров, маркетологов и контент-мейкеров. Одна из ключевых функций данного сервиса это то, что его функционал позволяет адаптировать цветовую палитру изображений с учетом семантического контекста, например, автоматически подбирая теплые тона для портретных изображений или интуитивно усиливает контраст городских пейзажей.

К преимуществам Adobe Firefly относится глубокая интеграция с экосистемой Creative Cloud – пользователи могут легко переносить проекты между Photoshop, Illustrator и Firefly, используя облачные библиотеки. Сервис сочетает скорость автоматической обработки с гибкостью ручной настройки, что делает его подходящим как для новичков, так и для профессионалов. ИИ модель демонстрирует высокое контекстное понимание, минимизируя типичные ошибки автоматизации, например, искажение текстур или цветовые артефакты. Кроме того, облачная инфраструктура Adobe позволяет обрабатывать изображения в высоком разрешении без нагрузки на локальное оборудование, что критически важно для работы с 4K-контентом.

Однако у сервиса есть ряд ограничений. Главный недостаток – зависимость от подписки Creative Cloud, стоимость которой начинается от \$20 в месяц, что делает Firefly малодоступным для индивидуальных пользователей или стартапов. Многие продвинутые функции, такие как работа со слоями или прецизионная настройка кривых, доступны только в десктопных версиях Photoshop, тогда как онлайн-интерфейс Firefly остаётся упрощённым. Кроме того, несмотря на заявленную простоту, достижение профессионального результата требует понимания основ работы с искусственным интеллектом (ИИ). Неправильно сформулированные текстовые запросы могут привести к неестественной цветопередаче или чрезмерной стилизации. В некоторых случаях алгоритмы генерируют артефакты, например, создают обильное количество шумов в тенях или неестественные градиенты, что требует дополнительной ручной доработки.

Исходя из вышеперечисленных преимуществ и недостатков сервиса Adobe Firefly можно сказать, что сервис представляет собой мощный инструмент для автоматизированной коррекции изображения. Однако повышенная стоимость, зависимость от экосистемы и недоступность в России оставляют нишу для специализированных и доступных решений, ориентированных на массового пользователя. Данный пример показывает важность баланса между автоматизацией ИИ и легким взаимодействием у пользователя.

1.2.3 Fotor (ИИ режимы)

Fotor – еще один онлайн-редактор изображений, который позволяет изменять изображения на основе технологий искусственного интеллекта. Сервис позиционирует себя как инструмент для быстрого ретуширования и стилизации фотографий (рисунок 1.4). Это делает его востребованным среди блогеров, СМИ и любителей, которые хотят улучшить свой контент без изучения различных средств обработки изображений и их сложные для начинающего пользователя настройки. Одной из ключевых особенностей Fotor является набор ИИ-режимов, которые включают в себя автоматическую цветокоррекцию, наличие готовых стилизованных фильтров и базовое улучшение качества фотографий.

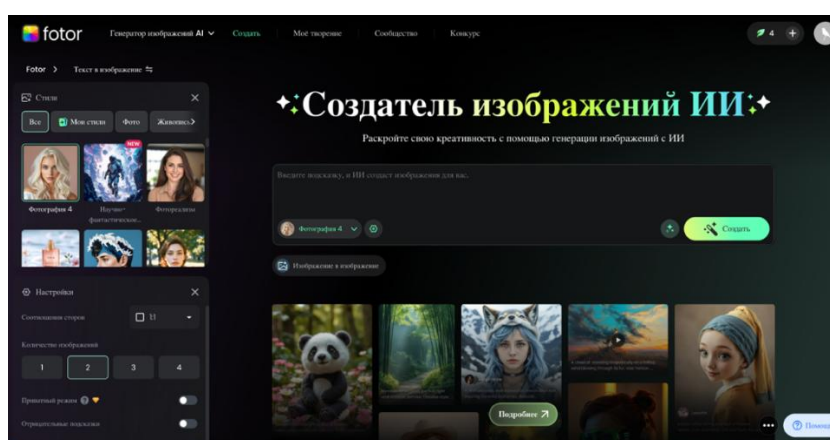


Рисунок 1.4 – Интерфейс Fotor

Технологической основой ИИ-режимов Fotor выступают предобученные свёрточные нейронные сети (рисунок 1.5), обученные на структурированных наборах данных (датасетах) с размеченными изображениями. Для цветокоррекции используется модель, анализирующая распределение цветовых каналов и сравнивающая его с «эталонными» гистограммами, характерными для профессионально обработанных фотографий [4, 8, 10].

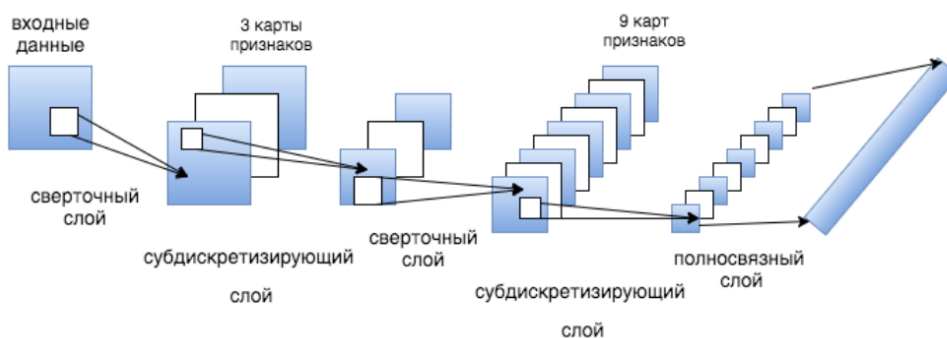


Рисунок 1.5 – Устройство свёрточной нейронной сети

Например, алгоритм стремится привести гистограмму к нормализованному виду, расширяя динамический диапазон, но без учёта семантики изображения. Это означает, что коррекция, подходящая для пейзажа, может быть некорректно применена к портрету, делая тон кожи неестественно оранжевым или бледным. Для стилизации применяются алгоритмы переноса стиля (style transfer), которые накладывают текстуры и паттерны на исходное изображение, однако их реализация упрощена по сравнению с аналогами вроде Prisma или DeepArt [12].

К преимуществам Fotor относится его доступность и простота интерфейса. ИИ-режимы работают быстро – обработка занимает не более 5–10 секунд, даже для изображений с высоким разрешением. Кроме того, Fotor предлагает бесплатный тариф с ограниченным функционалом, что привлекает аудиторию с небольшим бюджетом. Для пользователей, которым требуется больше возможностей, доступна подписка, включающая расширенные фильтры и удаление водяных знаков. Однако у сервиса есть значительные ограничения. Это касается профессиональной цветокоррекции. Главным недостатком является низкая гибкость настроек, которые пользователи не могут изменять. В особенности это касается отдельных цветовых каналов, кривых и масок.

Подводя итог по данному сервису, стоит отметить, что Fotor демонстрирует потенциал упрощенных решений в сфере искусственного интеллекта, которые могут быть полезны в рамках решения простых задач цветокоррекции, но никак не для задач профессиональной обработки.

1.3 Сравнительный анализ аналогов

Проведя обзор существующих решений для автоматизированной обработки, стоит выделить ключевые различия между всеми сервисами, которые могут предоставить цветокоррекцию графического изображения. Let's Enhance, Adobe Firefly и Fotor представляют три разных подхода к интеграции ИИ в обработку изображений, каждый из которых имеет свои сильные и слабые стороны.

Вариант доступа к сервису действительно является решающим фактором выбора пользователя, потому что чем проще добраться до функционала сервиса, тем больше пользователей будут заинтересованы в использовании данного сервиса. Let's Enhance и Fotor – это полностью онлайн-сервисы, которые не требуют установки дополнительно ПО, что делает их доступными для пользователя с любым уровнем знаний.

Используемые технологии искусственного интеллекта напрямую влияют на качество обработки. Let's Enhance опирается на генеративно-состязательные сети, что обеспечивает высокую детализацию при увеличении разрешения, но слабо адаптируется к семантике изображения. Например, при цветокоррекции алгоритм не различает портреты и пейзажи, применяя универсальные шаблоны. Adobe Firefly, благодаря гибридной архитектуре (диффузионные модели + трансформеры), демонстрирует глубокое понимание контекста: нейросеть распознает объекты и корректирует цвета выборочно, сохраняя естественность. Fotor, в свою очередь, использует упрощенные заранее обученные нейронные сети, которые анализируют только гистограммы, что приводит к поверхностной коррекции – например, избыточному усилению насыщенности без учёта композиционного устройства изображения [3, 4].

1.4 Выводы по разделу

Анализ существующих решений в области автоматизации цветокоррекции выявил как достижения, так и существенные ограничения современных сервисов. С одной стороны, такие платформы, как Let's Enhance, Adobe Firefly и Fotor, демонстрируют возможности нейросетевых алгоритмов в повышении качества изображений и упрощении рутинных задач. С другой стороны, каждая из них сталкивается с рядом проблем: отсутствие тонкой настройки и контекстного анализа (Let's Enhance, Fotor), высокая стоимость и сложность интеграции в рабочий процесс (Adobe Firefly), а также универсальность алгоритмов в ущерб точности и художественной выразительности.

Объединяющим фактором для всех рассмотренных решений является дисбаланс между автоматизацией и возможностью точного контроля, что ограничивает их применение в задачах, требующих гибкой и качественной цветокоррекции. При этом потребность в доступных и эффективных инструментах, сочетающих скорость ИИ и точность ручной обработки, продолжает расти – особенно среди пользователей, не обладающих профессиональными навыками работы с графическими редакторами.

Таким образом, можно сделать вывод о целесообразности разработки онлайн-сервиса, который будет учитывать как технические, так и эстетические аспекты цветокоррекции. Такой сервис должен обладать интуитивным интерфейсом, использовать продвинутые нейросетевые модели с семантическим анализом изображений и быть доступным широкой аудитории. Это определяет направление последующего исследования и разработки в рамках данной выпускной квалификационной работы.

ГЛАВА 2. ТЕОРИТИЧЕСКИЕ ВОПРОСЫ ПРОЕКТИРОВАНИЯ

2.1 Постановка задачи

Современные технологии искусственного интеллекта и машинного обучения находят широкое применение в области обработки изображений. Пользователи всё чаще ищут простые и доступные инструменты, позволяющие улучшать фотографии, применять художественные фильтры или стилизовать изображения без необходимости использовать сложные графические редакторы. На этом фоне особую актуальность приобретают интеллектуальные помощники, интегрированные в популярные мессенджеры, такие как Telegram. Telegram-боты предоставляют удобный способ взаимодействия, не требующий установки дополнительного программного обеспечения [7].

Целью данной выпускной квалификационной работы является разработка Telegram-бота, способного выполнять автоматическую и пользовательскую цветокоррекцию изображений, а также применять к ним нейросетевую стилизацию. Бот должен быть простым в использовании, устойчивым к пользовательским ошибкам и предоставлять визуальный отклик в понятной форме.

В рамках решения поставленной задачи реализуются следующие функции:

- 1) Приём изображений от пользователя (в виде фотографии или файла).
- 2) Автоматическая коррекция параметров изображения (яркость, контраст, насыщенность).
- 3) Применение заранее заданных пресетов (например, "винтаж", "кино").
- 4) Возможность ручной настройки параметров через кнопки управления.
- 5) Стилизация изображения с использованием генеративной нейросети Stable Diffusion по текстовому описанию.
- 6) Формирование коллажа «до/после» и предоставление пользователю результата.
- 7) Возможность скачать обработанное изображение или начать новую сессию.

Основная цель проекта – создание интуитивно понятного Telegram-бота, сочетающего проверенные классические методы обработки изображений (улучшение резкости, шумоподавление, цветокоррекция) с передовыми генеративными ИИ-моделями и позволяющего пользователям без технической подготовки получить качественные результаты. Система включает модуль загрузки и предварительной обработки, автоматически определяющий тип контента (портрет, пейзаж, документ и т. д.). Генеративный ИИ-модуль,

отвечающий за стилизацию под различные художественные направления (импрессионизм, акварель, киберпанк), восстановление утерянных деталей и добавление новых элементов «по контексту»; а также продуманный интерфейс использования (user experience - UX): простое меню в чате для выбора пресетов или тонкой настройки параметров, пошаговые подсказки и демонстрация «до/после», прогресс-бар обработки и передача готовых файлов через облако. Инфраструктура разворачивается локально с мониторингом производительности и логированием для быстрого реагирования на сбои. В результате пользователи получают доступный и мощный инструмент без установки сложного ПО и легко масштабируется при росте числа запросов.

2.2 Общая архитектура решения

Разрабатываемое программное решение представляет собой Telegram-бота, который взаимодействует с пользователем в режиме диалога и предоставляет функциональность по обработке изображений. Архитектура проекта построена на основе событийно-ориентированного подхода, где каждый этап обработки инициируется действиями пользователя: отправкой изображения, выбором способа коррекции или введением текста для нейросетевой стилизации [8, 13].

Система условно делится на три взаимосвязанных компонента:

Пользовательский интерфейс – реализован через Telegram-бота с использованием библиотеки Aiogram. Пользователь отправляет изображение и взаимодействует (рисунок 2.1) с ботом с помощью сообщений и встроенных кнопок в ряду (inline-кнопок). Вся логика реагирования на команды и выборы реализуется через обработчики(хендлеры), обрабатывающие входящие события.

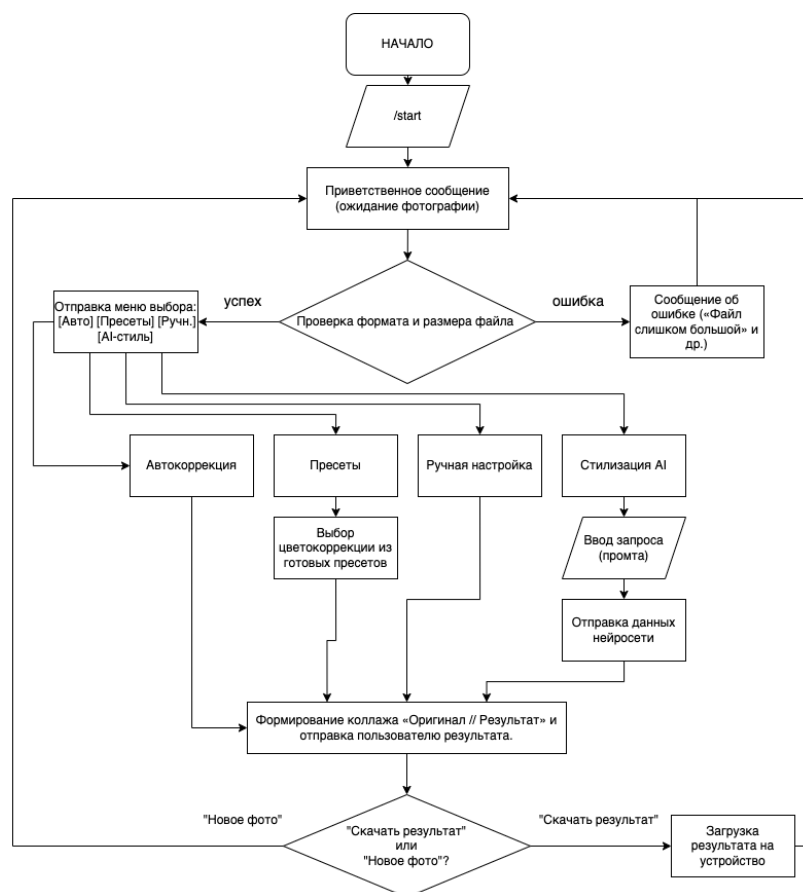


Рисунок 2.1 – Схема взаимодействия пользователя и онлайн сервиса

Модуль обработки изображений – включает в себя функции традиционной цветокоррекции, а также реализацию нейросетевой стилизации. Для

базовой коррекции используется библиотека Pillow, предоставляющая интерфейс для изменения яркости, контраста и насыщенности. Для нейросетевой стилизации применяется внешний API Replicate, предоставляющий доступ к модели Stable Diffusion, принимающей на вход изображение и текстовый запрос (промпт) [8, 12].

Хранение состояния и данных – осуществляется в оперативной памяти с помощью глобального словаря `user_images`, где для каждого пользователя временно сохраняются оригинальные и обработанные изображения. Это позволяет обеспечить независимость сессий разных пользователей и реагировать на команды в рамках контекста конкретного изображения.

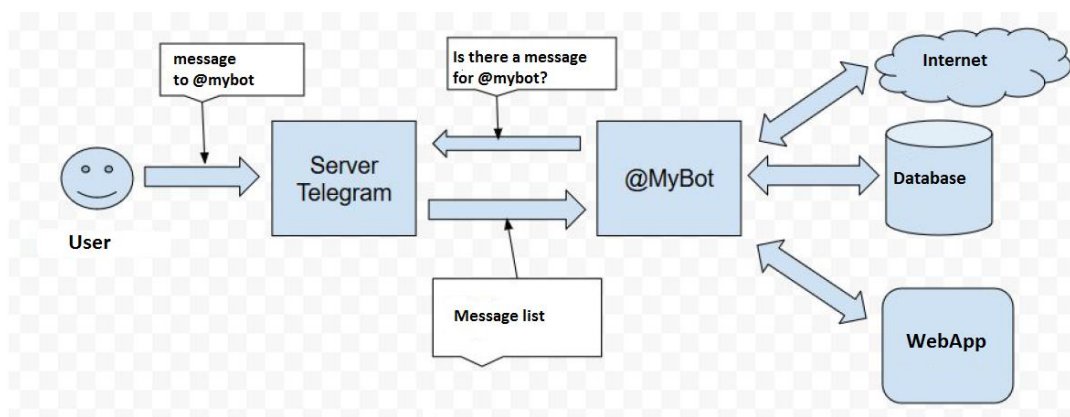


Рисунок 2.2 – Схема взаимодействия сервера Telegram и бота

Взаимодействие Telegram-бота с серверами Telegram (рисунок 2.2). Telegram-боты взаимодействуют с сервером Telegram через Bot API – HTTP-интерфейс, предоставляемый Telegram для управления ботами. Код бота на Python (с использованием библиотек, например, «python-telegram-bot» или «aiogram») отправляет запросы к этому API и обрабатывает ответы. Рассмотрим процесс детально.

Аутентификация и подключение Telegram-бота к серверам Telegram являются фундаментальными этапами его работы, обеспечивающими безопасное и корректное взаимодействие с платформой. Процесс начинается с получения уникального токена бота, который выступает в роли цифрового ключа для доступа к Bot API. Токен создается автоматически при регистрации бота через официальный бот Telegram @BotFather. Он имеет формат «123456:ABCDEF1234ghIkl-zyx57W2v1u123ew11», где первая часть представляет идентификатор бота, а вторая – секретный ключ, гарантирующий, что запросы отправляются именно владельцем бота. Этот токен необходимо хранить в безопасности, например, в переменных окружениях или зашифрованных

конфигурационных файлах, чтобы избежать утечки и несанкционированного доступа к функционалу бота.

Каждое взаимодействие бота с серверами Telegram происходит через HTTPS-запросы к Bot API, в URL которых обязательно включается токен. Например, для вызова метода `getMe`, который возвращает основную информацию о боте, формируется запрос:

```
https://api.telegram.org/bot<токен>/getMe
```

2.2.1 Пользовательский интерфейс

Пользовательский интерфейс реализуется с помощью Telegram-бота, написанного на языке Python с использованием асинхронной библиотеки Aiogram. Эта библиотека предоставляет высокоуровневый инструмент для создания Telegram-ботов, построенных на событийной модели обработки. Взаимодействие пользователя с ботом осуществляется через текстовые команды, отправку сообщений/изображений в чат, кнопки интерфейса Telegram.

Текстовые команды – это базовый способ управления ботом, с которого начинается работа с системой. Данная команда описывается в файле с обработчиками данных (листинг 2.2). В Telegram такие команды вводятся вручную (например, /start) или вызываются через интерфейс команд самого Telegram.

В рамках проекта реализована команда /start, с помощью которой бот:

- 1) Приветствует пользователя;
- 2) Объясняет, что он умеет;
- 3) Предлагает отправить изображение для дальнейшей работы.

Листинг 2.2 – Взаимодействие с командой /start (handlers.py)

```
@dp.message(Command("start"))
async def send_welcome(message: types.Message):
    await message.reply(
        "Привет! Я бот для цветокоррекции и стилизации изображений. 🌈\n"
        "Отправь мне фото (как изображение или файл), и я его улучшу!"
    )
```

Это событие служит точкой входа в пользовательский сценарий. Оно не только объясняет функциональность бота, но и переводит пользователя в состояние готовности – т.е. ожидания изображения.

Отправка изображений в чат (в виде файла или фотографии) в Telegram изображения могут быть переданы двумя способами: как фотография, сжатая до Telegram-формата (менее качественная, но быстрая); как документ, то есть оригинальный файл (без потери качества, но больше размер).

Бот поддерживает оба варианта. При получении изображения бот: проверяет формат и размер; извлекает байтовый поток изображения; преобразует его в объект PIL.Image; сохраняет результат в словарь user_images;

Для взаимодействия с интерфейсом онлайн сервиса были использованы внутренние возможности мессенджера и реализованы кнопки, нажав на которые пользователю будет предложено воспользоваться навыками сервиса. Созданы кнопки (листинг 2.3): Автокоррекция, Пресеты, Ручная настройка, Стилизация AI. Реализация кнопок в подробном варианте на языке Python представлена в листинге 2.3.

Листинг 2.3 – Реализация кнопок на языке Python(handlers.py)

```
keyboard = InlineKeyboardMarkup(inline_keyboard=[
    [
        InlineKeyboardButton(
            text="Автокоррекция", callback_data="auto"),
        InlineKeyboardButton(text="Пресеты", callback_data="pre-
sets"),
    ],
    [
        InlineKeyboardButton(text="Ручная настройка",
callback_data="manual"),
        InlineKeyboardButton(text="Стилизация (AI) ",
callback_data="ai_style"),
    ]
])

await message.reply("Выберите тип коррекции:", reply_markup=key-
board)
```

Особенностями реализации являются: проверка типа MIME и расширения файла обеспечивает безопасность, поддержка документов позволяет работать с изображениями до 10 МБ без потери качества. использование библиотеки PIL.Image позволяет гибко обрабатывать изображения дальше. Данный метод является ключевым методом решения проблемы загрузки изображения для его дальнейшей обработки сервисом.

Нажатие кнопок, которые формируют логическую структуру интерфейса выбора, после загрузки изображения пользователь получает inline-клавиатуру (рисунок 2.3) – набор кнопок, встроенных прямо в сообщение бота. Они позволяют выбрать тип обработки без ввода текста вручную.

Пример интерфейса:

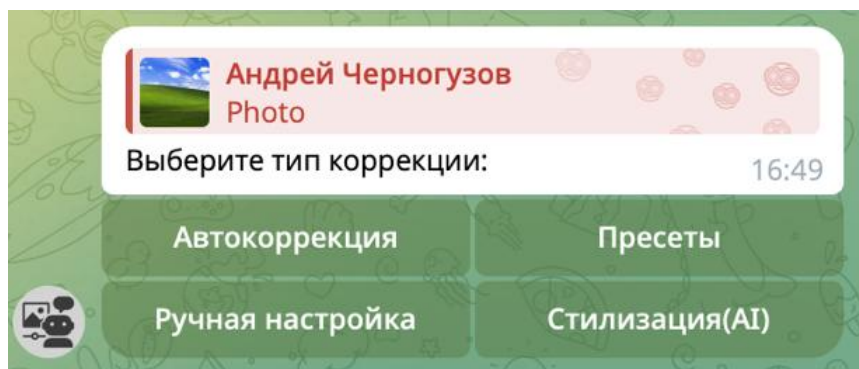


Рисунок 2.3 – Интерфейс взаимодействия с ботом.

Пользователь нажимает кнопку – бот получает `callback_query`, из которого извлекается `callback_data` и выполняет соответствующее действие. Данное событие описано в листинге 2.4, в котором происходит обработка кнопок, с которыми будет взаимодействовать пользователь.

Обработка нажимаемых кнопок происходит в функции:

Листинг 2.4 – Обработка кнопок в @dp.callback_query() (handlers.py)

```
@dp.callback_query()
async def process_buttons(callback_query: types.CallbackQuery, state: FSMContext):
    user_id = callback_query.from_user.id
    data = callback_query.data
```

В зависимости от полученного значения в поле `data` выполняется либо автоматическая цветокоррекция, либо открывается подменю с пресетами, либо настраивается ручное управление (яркость/контраст), либо активируется ожидание текстового описания для AI-стилизации.

Пользовательский интерфейс бота выстроен таким образом, чтобы каждый этап работы с изображением был максимально предсказуемым и понятным даже для тех, кто никогда раньше не сталкивался с графическими редакторами. При запуске бот сразу сообщает о своих возможностях и приглашает отправить фотографию – это позволяет переключить фокус пользователя с команд и синтаксиса на сам контент. Как только изображение поступает, бот подтверждает факт приёма и моментально показывает меню с кнопками: «Автокоррекция», «Пресеты», «Ручная настройка», «Стилизация (AI)». Такое меню не перегружено – всего четыре варианта, каждый из которых подписан максимально просто и понятно.

Нажатие на кнопку служит чётким сигналом для системы: пользователь не вводит ничего вручную, а выбирает готовый вариант. При выборе «Автокоррекция» бот делает всё сам и сразу выдаёт результат, что даёт ощущение «одного клика и готово». Если же нажать «Пресеты», открывается второе подменю с двумя стилями – «Винтаж» и «Кино» – и кнопкой «Назад» для возврата в главное меню. Это вложенное меню реализовано так, чтобы не нагружать главный экран сразу всеми вариантами, но при этом дать расширенные возможности тем, кто хочет чуть больше контроля.

Режим «Ручная настройка» демонстрирует ещё более тонкий подход: три пары кнопок «+/-» для яркости и контраста, а также кнопка «Готово», позволяющая завершить настройку и вернуться назад (листинг 2.5). Здесь важно, что после каждого нажатия бот мгновенно обновляет изображение, чтобы пользователь видел эффект своих действий – это ключ к хорошему UX в интерактивных системах. Поэтому основой для создания бота был выбран Telegram.

Листинг 2.5 – Создание клавиатуры выбора (handlers.py)

```
elif data == "manual":
    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="+ Яркость",
            callback_data="brightness_up"),
```

Продолжение листинга 2.5

```
        InlineKeyboardButton(text="- Яркость",
callback_data="brightness_down")],
[InlineKeyboardButton(text="+ Контраст", callback_data="contrast_up"),
    InlineKeyboardButton(text="- Контраст", callback_data="con-
trast_down")],
    [InlineKeyboardButton(text="Готово", callback_data="back")]
])
    await callback_query.message.edit_reply_markup(reply_markup=key-
board)
```

Особая роль отводится режиму «Стилизация (AI)»: при выборе этой опции бот не сразу начинает работу, а переводит диалог в состояние ожидания текстового описания. Пользователь получает понятное приглашение «Введите текстовое описание для стилизации», после чего любая фраза воспринимается как промпт для нейросети (листинг 2.6). Это позволяет избежать путаницы и гарантирует, что бот не будет пытаться обработать не тот ввод.

Листинг 2.6 – Запрос промпта у пользователя(handlers.py)

```
elif data == "ai_style":
    await callback_query.answer()
    await callback_query.message.answer("Введите текстовое описание
для стилизации:")
    await state.set_state(Form.waiting_for_prompt) # Устанавливаем
состояние ожидания
    return
```

В завершение каждого процесса бот формирует «коллаж» из двух частей – «до» и «после» – и отправляет его с подписью, поясняющей, где оригинал, а где результат. Под изображением всегда присутствуют кнопки «Скачать результат» и «Обработать другое фото», так что пользователь может сохранить файл или сразу начать новый сеанс. Весь диалог оформлен в нейтральных тонах, без излишней технической терминологии, что делает взаимодействие комфортным и понятным даже новичку.

Данная архитектура обеспечивает модульность, расширяемость и устойчивость системы. Разделение логики взаимодействия с пользователем и логики обработки изображений позволяет легко масштабировать проект: добавлять новые методы коррекции, поддержку других форматов или интеграцию дополнительных нейросетей.

2.3 Используемые технологии и библиотеки

В ходе разработки Telegram-бота для цветокоррекции изображений была выбрана комбинация легковесных и проверенных временем инструментов, позволяющих быстро реализовать как базовые функции обработки, так и интеграцию с современными нейросетями. Ниже перечислены основные технологии и библиотеки, использованные в проекте, с указанием их ролей и преимуществ.

2.3.1 Python

Python – это высокоуровневый, интерпретируемый язык программирования общего назначения, созданный для быстрого прототипирования и удобного написания кода приложения(кодинга). Он сочетает в себе «читаемый» синтаксис с динамической типизацией и богатым набором встроенных структур данных, что позволяет разработчикам сосредоточиться на логике решения задач, а не на деталях управления памятью или сложном синтаксисе. Благодаря интерпретируемой природе и поддержке объектно-ориентированного, процедурного и функционального стилей программирования, Python отлично подходит как для небольших утилит и скриптов, так и для крупных многомодульных приложений [13].

Интерпретатор Python и обширная стандартная библиотека распространяются в виде исходного кода или готовых бинарных сборок для всех популярных платформ и могут свободно использоваться и распространяться. Стандартная библиотека включает модули для работы с сетью, файловой системой, базами данных, Web-программирования, многопоточности и многого другого, что значительно ускоряет разработку и уменьшает зависимость от сторонних компонентов.

Благодаря поддержке расширений на C и C++ (а также интеграции с другими языками через механизмы FFI), Python легко встраивается в существующие системы и может выступать «клеевым» языком, объединяя высокопроизводительные компоненты, написанные на компилируемых языках, с гибкой бизнес-логикой на Python. Именно такая философия расширяемости и открытости лежит в основе успеха языка и его широкой экосистемы.

Одним из главных преимуществ языка Python является его лаконичный и интуитивно понятный синтаксис, который во многих случаях читается почти как псевдокод. Это делает Python особенно привлекательным для начинающих разработчиков и существенно снижает порог входа в программирование. При этом язык остаётся мощным инструментом и для опытных инженеров, позволяя решать задачи промышленного уровня сложности. Python не требует декларации типов, обладает автоматическим управлением памятью и поддерживает динамическую типизацию, что делает процесс написания и тестирования кода более быстрым и гибким по сравнению с языками со строгой типизацией, такими как Java или C++.

Одной из ключевых особенностей Python является его огромная экосистема. Тысячи сторонних библиотек и фреймворков позволяют использовать

Python практически в любой сфере: от веб-разработки (Django, Flask) и научных расчётов (NumPy, SciPy, Matplotlib) до машинного обучения (TensorFlow, PyTorch, scikit-learn) и автоматизации (Selenium, Paramiko). Это делает Python универсальным языком, который одинаково хорошо подходит как для написания веб-приложений, так и для реализации прототипов нейросетей, серверных бэкендов или программ автоматизации.

Python также отличается кроссплатформенностью – программы, написанные на этом языке, работают на Windows, Linux, macOS и даже встраиваемых системах с минимальными изменениями или вовсе без них. Это удобно как в разработке, так и при развёртывании решений на разных операционных системах. Кроме того, Python активно используется в сфере DevOps и Data Science благодаря удобству работы с файлами, базами данных, REST API, а также возможности быстрой визуализации и анализа данных.

Язык имеет очень активное сообщество и развитую документацию. Это означает, что почти на любую проблему можно найти готовое решение, учебный материал или поддержку в профессиональной среде. Python также популярен в академической среде, что делает его часто используемым инструментом в научных проектах и стартапах, где важны скорость разработки и доступность готовых решений.

В сравнении с другими языками программирования, такими как Java, C++ или JavaScript, Python выигрывает по критерию читаемости кода и скорости разработки. Его интерпретируемая природа может делать его менее производительным в вычислительно интенсивных задачах, однако в большинстве прикладных сценариев это компенсируется простотой реализации и богатой библиотечной поддержкой. Кроме того, при необходимости критические участки кода могут быть переписаны на C или Cython для повышения производительности [13].

Таким образом, Python представляет собой сбалансированный язык, сочетающий простоту, гибкость и мощь. Это делает его идеальным выбором для быстрой разработки Telegram-ботов, интеграции с внешними API, обработки изображений и создания ИИ-сервисов – именно таких, какие реализуются в рамках данного проекта.

2.3.2 Aiogram

Aiogram – современная асинхронная библиотека для разработки Telegram-ботов на языке Python, построенная на базе стандарта «asyncio». Основная идея Aiogram заключается в разделении логики обработки входящих сообщений на независимые обработчики (хендлеры), каждый из которых реагирует на конкретные события: команды, текстовые сообщения, вложения, нажатия inline-кнопок и изменения состояний. За счёт асинхронной архитектуры бот, написанный на Aiogram, способен одновременно обслуживать множество пользователей, не блокируя основной поток выполнения при ожидании ответов от Telegram API или при выполнении внешних операций ввода-вывода, таких как загрузка изображений или запросы к внешним сервисам [13].

Одним из ключевых компонентов библиотеки является механизм фильтров: разработчик может задать гибкие условия, по которым определённые хендлеры будут активироваться только для нужных типов событий (например, только для фотографий, только для документов или только для команд). Это позволяет построить чёткую и предсказуемую систему маршрутизации сообщений, избавляя от необходимости вручную проверять типы входящих данных внутри кода.

Aiogram также включает встроенные средства для работы с клавиатурами и кнопками различного типа – как клавиатурами с возвращаемым ответом (reply-клавиатуры), так и с inline-клавиатурами – что упрощает создание интерактивного интерфейса для пользователя. Кнопки могут быть снабжены возвращаемыми данными, а нажатие на них обрабатывается отдельными хендлерами, что позволяет реализовать многоуровневые меню и сложные сценарии взаимодействия без избыточных проверок.

Не менее важной частью Aiogram является встроенная поддержка конечных автоматов (Finite State Machine, FSM). С помощью FSM можно разделить диалог с пользователем на последовательные шаги и организовать ожидание ввода от пользователя в различных контекстах – например, ожидание текстового промпта после выбора соответствующей опции. Это особенно актуально для многошаговых операций, когда важно сохранять контекст между разными сообщениями и корректно обрабатывать порядок действий.

Кроме того, библиотека предусматривает возможности для расширения через промежуточные слои (middleware), в которых можно выполнять глобальные проверки, логирование, аутентификацию или другие общие задачи до или

после работы хендлера. Такая модульная структура делает код более чистым и удобным для сопровождения.

Благодаря хорошо продуманному API, подробной документации и активному сообществу, Aiogram является одним из самых популярных инструментов для создания Telegram-ботов на Python. Он сочетает в себе простоту настройки, высокую производительность и широкие возможности по организации диалоговой логики, что позволяет быстро реализовать как простые ботов-ассистентов, так и сложные многофункциональные сервисы.

2.3.3 Pillow (PIL-Fork)

Pillow – это широко используемая библиотека для обработки растровых изображений в Python, созданная как продолжение оригинального проекта Python Imaging Library (PIL). Она предлагает удобный и понятный интерфейс для работы с изображениями: их загрузки, сохранения и преобразования в различных форматах, таких как PNG, JPEG, BMP, GIF, TIFF, WEBP и многих других. Поддержка разнообразных форматов позволяет разработчикам легко принимать изображения от пользователей и возвращать их в нужном виде без дополнительных инструментов или ручной обработки.

Одна из сильных сторон Pillow – модуль ImageEnhance, который упрощает базовую цветокоррекцию. С его помощью можно регулировать яркость, контраст и насыщенность (цветовой баланс). Эти операции выполняются быстро и эффективно, с минимальной нагрузкой на память, что делает библиотеку подходящей для приложений, работающих в реальном времени, таких как Telegram-боты. Можно создавать готовые стили (пресеты) или настраивать параметры вручную, сразу показывая пользователю результат [14].

Pillow также предоставляет инструменты для трансформации изображений: вращение, обрезка, изменение размеров, переключение цветовых пространств (например, из RGBA в RGB) и объединение нескольких картинок в одну – например, для создания коллажа «до/после». Поддержка байтовых потоков через BytesIO позволяет обрабатывать данные прямо в памяти, без сохранения на диск, что ускоряет и упрощает работу.

По сравнению с более сложными библиотеками, такими как OpenCV, Pillow выигрывает в простоте установки, отсутствии внешних зависимостей и скорости выполнения базовых задач. Это делает его идеальным выбором для веб-приложений и ботов. К тому же, благодаря доступной документации и поддержке сообщества, освоить Pillow несложно даже новичкам. В данном проекте он стал основой для цветокоррекции и обработки изображений, обеспечивая удобство и эффективность.

В проекте Pillow используется на всех этапах работы с изображениями: от получения файлов до создания итогового коллажа. В файле handlers.py импортируются основные модули: «from PIL import Image, ImageEnhance», что задаёт фундамент для обработки растровых данных.

Когда пользователь загружает изображение, бот принимает его как байтовый поток и открывает с помощью «Image.open(BytesIO(...))». Затем проверяется цветовой режим (свойство image.mode): если это RGBA или LA,

изображение преобразуется в RGB через создание белого фона и метод `paste`, чтобы правильно обработать прозрачность.

Функция `apply_color_correction` реализует ключевую логику обработки. Она использует три объекта `ImageEnhance` – для яркости (`Brightness`), контраста (`Contrast`) и насыщенности (`Color saturation`) – и изменяет параметры с помощью метода `enhance()`. Здесь же заложена основа для автоматической коррекции и пресетов: например, для стиля «cinematic» дополнительно усиливается контраст. В будущем функционал можно расширить, добавив работу с отдельными цветовыми каналами через доступ к пикселям.

После обработки оригинальное и исправленное изображения объединяются в коллаж. Новый холст создаётся через «`Image.new('RGB', (width*2, height))`», а метод `paste()` размещает на нём обе версии картинки. Итог сохраняется в буфер `BytesIO` в формате PNG и отправляется пользователю через Telegram API.

В конечном итоге, Pillow обеспечивает весь процесс обработки изображений – от загрузки до финального вывода – сочетая простоту, скорость и надёжность. В проекте он стал незаменимым инструментом для цветокоррекции и базового редактирования.

2.3.4 NumPy

В проекте библиотека NumPy выступает вспомогательным инструментом для работы с массивами пикселей и прототипирования более сложных алгоритмов цветокоррекции. Хотя основная обработка выполняется через Pillow, NumPy предоставляет низкоуровневый доступ к данным изображения и позволяет легко реализовывать операции над отдельными каналами или совокупными трансформациями целых массивов.

NumPy представляет собой фундаментальную библиотеку для научных вычислений в Python, позволяющую эффективно хранить и обрабатывать многомерные данные с помощью оптимизированных массивов (ndarray). Благодаря реализации на C и Fortran, большинство операций над массивами выполняется без явных циклов на Python (векторизованно), что обеспечивает высокую производительность даже при обработке больших объёмов пиксельной информации [9].

В коде проекта NumPy упоминается в разделе с пресетами в функции «apply_color_correction». Хотя сейчас реализация пресета «винтаж» закомментирована, она демонстрирует, как можно напрямую модифицировать каналы изображения через NumPy-массив (листинг 2.7).

Листинг 2.7 – Использование NumPy массива (handlers.py)

```
if preset == "vintage":  
    image = np.array(image)
```

Такая схема позволяет экспериментировать с любыми математическими операциями над пикселями – например, добавлять шум, применять гамма-коррекцию, изменять баланс цветовых каналов или реализовывать фильтры на основе линейных и нелинейных преобразований.

Кроме того, применение NumPy открывает путь для интеграции более сложных алгоритмов: от преобразования цветовых пространств (YCbCr, LAB) до реализации собственных свёрточных операций и эффектов, которые затем могут быть упакованы в пресеты. В будущем использование NumPy может быть расширено и для работы с масками, наложений и других визуальных эффектов, требующих манипуляций с массивами пикселей на низком уровне.

Таким образом, NumPy в проекте служит «мостом» между высокоуровневой обработкой Pillow и возможностями низкоуровневой матричной арифметики, обеспечивая гибкость и расширяемость цветокоррекционных пресетов.

2.3.5 Replicate API и Stable Diffusion (SDXL)

Replicate – это облачная платформа для запуска машинно-обученных моделей через HTTP API, позволяющая использовать мощные AI-модели без забот об инфраструктуре. Сервис предоставляет единый REST-интерфейс, через который можно запускать любые публичные открытые (open-source) модели (текстовые LLM, модели генерации изображений, аудио и др.). Пользователь может либо воспользоваться уже размещёнными моделями (например, Flux Schnell и др.), либо самостоятельно выгрузить и опубликовать свои модели для дообучения или генерации. Разработчики взаимодействуют с платформой через HTTP-запросы, передавая параметры модели и данные через API. Например, для запуска предсказания посылается POST-запрос на «<https://api.replicate.com/v1/predictions>» с указанием идентификатора версии модели и входных данных (таких как поле prompt для SDXL). Все запросы требуют указания API-токена в заголовке «Authorization: Bearer <token>», который генерируется в личном кабинете на сайте Replicate. Таким образом, Replicate служит универсальным интерфейсом к множеству моделей: он упрощает запуск предсказаний, обучение моделей, а также обеспечивает масштабирование и оплату за ресурсы «из коробки» [4, 8, 12].

Stable Diffusion XL (SDXL) – это новейшая версия генеративной модели для преобразования текста в изображение от Stability AI. SDXL основан на концепции латентной диффузии: сначала текстовый запрос кодируется трансформером (например, CLIP или OpenCLIP) в вектор вложения (вектор эмбедингов), затем в пространство низкоразмерного скрытого объекта (латента) (кодера-автоэнкодера) создаётся начальный шум, который последовательно очищается (де-ноизится) U-Net’ом, условленным на текстовый эмбединг. Наконец, полученные «очищенные» латенты передаются через декодер автоэнкодера для получения пиксельного изображения. В SDXL используется многократный диффузионный процесс: на каждом шаге U-Net добавляет и удаляет шум по заданному расписанию, последовательно улучшая качество изображения. Как и в предыдущих версиях Stable Diffusion, трансформерные слои с механизмом сосредоточения одного типа данных на другом (кросс-аттеншена) внедряют информацию из текстового эмбединга на различных масштабах изображения.

Важное отличие SDXL от Stable Diffusion 1.5 и 2.x – масштаб и состав модели. У U-Net SDXL примерно 2.6 млрд параметров, что почти в три раза больше, чем у SD1.5/2.1 (≈860 млн). Архитектура U-Net также переработана: в

отличие от равномерного распределения блоков в старых версиях, в SDXL используется гетерогенное распределение трансформер-блоков по уровням (например, отсутствуют блоки на самом высоком уровне, увеличено их число на средних уровнях). Ключевым нововведением стали двойные текстовые энкодеры: SDXL объединяет выходы OpenCLIP ViT-bigG и CLIP ViT-L, расширяя «контекст» текстового эмбединга до 2048 элементов. В старых SD 1.5 использовался один CLIP ViT-L/14 (контекст 768), а в SD2.1 – один OpenCLIP ViT-H (контекст 1024). Такое дублирование энкодеров даёт более богатое текстовое условие и повышает детализацию. В совокупности это увеличивает и общий размер модели: текстовые энкодеры SDXL имеют ≈ 817 млн параметров дополнительно. Также в SDXL применён улучшенный автоэнкодер (вариационный), обученный с большим объёмом данных, что повышает точность восстановления деталей изображения [8, 12].

SDXL вводит несколько инноваций по условию генерации и этапам пост-обработки. Модель тренируется на изображениях разных соотношений сторон и учитывает исходный размер картинки через механизм «size-conditioning»: высота и ширина кодируются и добавляются в модель в качестве дополнительного условия. При генерации это позволяет получать изображения высокого разрешения и нужного соотношения без потери обучающей информации. Наконец, в SDXL 1.0 используется двухэтапный конвейер генерации: сначала базовая модель генерирует «грубые» латенты, затем отдельная специальная модель-уточнитель (refinement model) выполняет финальную доочистку (SDEdit) и доводит изображение до фотореалистичного качества. Согласно официальному анонсу Stability AI, SDXL 1.0 состоит из базовой модели (≈ 3.5 млрд параметров) и уточняющей (в совокупности ≈ 6.6 млрд), благодаря чему эта система «бьёт» по качеству все предыдущие открытые модели. Пользовательские тесты подтвердили значительный превосходство SDXL над SD1.5 и SD2.1 по качеству изображений.

2.3.6 Aiogram-FSM

Aiogram-FSM – это встроенный в Aiogram механизм управления конечными автоматами (Finite State Machine, FSM), обеспечивающий чистую и надёжную реализацию многошаговых диалоговых сценариев внутри Telegram-бота. В традиционных подходах к разработке ботов для каждого нового этапа взаимодействия приходилось вручную отслеживать контекст и проверять, на каком шаге находится пользователь, что приводило к разрастанию условных операторов и путанице в коде. FSM же формализует этот процесс: каждый пользователь получает «состояние», определяющее, какие сообщения и команды он может отправить и как на них реагировать.

При использовании Aiogram-FSM разработчик объявляет набор состояний (обычно через подкласс `StatesGroup`), а затем связывает конкретные хендлеры с одним или несколькими состояниями. Когда бот переводит пользователя в новое состояние, все последующие сообщения этого пользователя автоматически фильтруются и обрабатываются только теми хендлерами, которые ожидают именно это состояние. Это позволяет, во-первых, избежать избыточных проверок внутри функций, и, во-вторых, сохранять ясную логику переходов между шагами.

В контексте проекта с цветокоррекцией и стилизацией FSM применяется для режима «Стилизация (AI)»: после нажатия соответствующей кнопки бот переводит пользователя в состояние ожидания текстового описания. Это гарантирует, что первый же текст, присланный пользователем, будет обработан как промпт для нейросети, а не как новая команда или случайный ввод. По завершении стилизации состояние очищается, и пользователь возвращается в основное меню, готовым к новому циклу работы [13].

Кроме того, FSM упрощает реализацию сложных ветвлений сценариев. Можно легко задавать «подсостояния» для настройки нескольких параметров одновременно, объединять состояния в группы и организовывать переходы «назад» и «в начало» без повторного описания логики. Встроенные методы Aiogram-FSM позволяют сохранять и извлекать данные состояния – например, промежуточные результаты обработки или пользовательские параметры прямо в контексте сессии. Это делает код более модульным, облегчает тестирование отдельных шагов диалога и снижает вероятность ошибок при расширении функционала бота.

2.4. Обработка изображений: методы и алгоритмы

В основе функциональности Telegram-бота лежит несколько взаимодействующих методов обработки изображений, реализованных с помощью библиотек Pillow и NumPy, а также внешнего API Replicate для нейросетевой стилизации. Ниже подробно рассмотрены алгоритмы цветокоррекции, применение пресетов, ручная настройка и создание итогового коллажа [14].

2.4.1 Автоматическая цветокоррекция

Автоматическая цветокоррекция представляет собой последовательное усиление ключевых параметров изображения – яркости, контраста и насыщенности. Для каждого параметра используется класс `ImageEnhance` из `Pillow` (листинг 2.8):

Яркость (`Brightness`) – создаётся объект усилителя, который увеличивает уровень светлоты всех пикселей в изображении на заданный коэффициент (в коде по умолчанию 1.5).

Контраст (`Contrast`) – следующий усилитель повышает разницу между светлыми и тёмными участками (коэффициент 1.5).

Насыщенность (`Color`) – последний усилитель усиливает цветовой баланс, делая оттенки более интенсивными (коэффициент 1.2).

Листинг 2.8 – Реализация автоматической цветокоррекции (`handlers.py`)

```
def apply_color_correction(image, preset=None, brightness=1.5, contrast=1.5,
saturation=1.2):
    enhancer = ImageEnhance.Brightness(image)
    image = enhancer.enhance(brightness)

    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(contrast)

    enhancer = ImageEnhance.Color(image)
    image = enhancer.enhance(saturation)
```

Такой простой конвейер обеспечивает сбалансированное улучшение картинки без артефактов и подходит для большинства фотографий, требующих базового освежения и коррекции экспозиции [1, 2, 14].

2.4.2 Пресеты цветовых стилей

Помимо универсальной авто-коррекции, реализована система пресетов – заранее заданных конфигураций усилителей, придающих изображению характерные художественные оттенки. В текущей версии доступны 7 пресетов (листинг 2.9):

«Винтаж» – предполагает работу на уровне отдельных цветовых каналов через NumPy-массив изображения: зелёный канал слегка уменьшался (комментарий в коде), создавая тёплый ретро-эффект.

«Кино» – усиливает контраст сильнее, чем в авторежиме (коэффициент 1.8), что придаёт картинке более драматичный, «кинематографический» вид.

Каждый пресет дополняет базовую цепочку усилителей и может быть расширен в будущем за счёт дополнительных операций: кривых тональных коррекций, добавления зерна или цветовых фильтров через линейные преобразования массивов пикселей [5, 6].

Листинг 2.9 – Создание готовых пресетов для обработки изображения (handlers.py)

```
if preset == "vintage":
    image = np.array(image)
    image[:, :, 1] = image[:, :, 1] * 0.8
    image = Image.fromarray(np.uint8(image))
elif preset == "cinematic":
    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(0.7)
elif preset == "enhanced_contrast":
    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(1.4)
elif preset == "bw":
    image = np.array(image)
    gray = np.mean(image, axis=2, keepdims=True)
    image = np.repeat(gray, 3, axis=2).astype(np.uint8)
    image = Image.fromarray(image)
elif preset == "red_only":
    image = np.array(image)
    image[:, :, 1] = 0 # Обнуляем зеленый
    image[:, :, 2] = 0 # Обнуляем синий
    image = Image.fromarray(np.uint8(image))
elif preset == "blue_only":
    image = np.array(image)
    image[:, :, 0] = 0 # Обнуляем красный
    image[:, :, 1] = 0 # Обнуляем зеленый
    image = Image.fromarray(np.uint8(image))
elif preset == "warm_tone":
    image = np.array(image)
    image[:, :, 0] = image[:, :, 0] * 1.3 # Усиливаем красный
    image[:, :, 2] = image[:, :, 2] * 0.7 # Уменьшаем синий
    image = Image.fromarray(np.uint8(image))
return image
```


2.4.3 Коллаж «до/после»

Независимо от способа коррекции, финальный результат подаётся в виде единого изображения (листинг 2.10), где слева – оригинал, а справа – скорректированная версия. Алгоритм создания коллажа следующий:

1. Определяется размер исходного изображения ($\text{width} \times \text{height}$).
2. Создаётся новый холст `Image.new('RGB', (2·width, height))`.
3. Методом `paste()` вставляются два изображения рядом.

Коллаж сохраняется в память через BytesIO и передаётся Telegram API как единое фото.

Такой формат сравнения позволяет пользователю мгновенно оценить эффект обработки [3, 11].

Листинг 2.10 – Реализация создания и отправки коллажа «до/после» в функции `send_result()` (`handlers.py`)

```
width, height = original_image.size
collage = Image.new('RGB', (width * 2, height))
collage.paste(original_image, (0, 0))
collage.paste(corrected_image, (width, 0))

with BytesIO() as buffer:
    collage.save(buffer, format="PNG")
    buffer.seek(0)

    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="Скачать результат",
callback_data="download")],
        [InlineKeyboardButton(text="Обработать другое фото",
callback_data="backmenu")]
    ])

    await callback_query.message.answer_photo(
        types.BufferedInputFile(buffer.getvalue(), filename="re-
sult.png"),
        caption="Слева - оригинал, справа - результат",
        reply_markup=keyboard
    )
    await callback_query.answer()
except Exception as e:
    await callback_query.message.answer(f"Ошибка при создании результата:
{str(e)}")
    await callback_query.answer()
```

2.4.4 Нейросетевая стилизация (AI Style)

Для более творческой стилизации бот использует модель Stable Diffusion XL через облачный API Replicate (листинг 2.11). Пользователь вводит текстовый промпт, описывающий желаемый художественный стиль или сцену. Алгоритм работы:

Исходное изображение преобразуется в байтовый поток и вместе с текстом отправляется в «replicate.run» с указанием модели «stability-ai/sdxl» и параметра «strength» (по умолчанию 0.7) [12].

Replicate генерирует серию латентных представлений, очищая шум в соответствии с текстовым эмбедингом, и возвращает URL итогового изображения.

Бот загружает результат по полученному адресу и сохраняет его как PIL.Image.

Готовое изображение также объединяется с оригиналом в коллаж «до/после» и отправляется пользователю.

Листинг 2.11 – Реализация чтения и отправки запроса пользователя к нейросети (handlers.py)

```
original_image = user_images[user_id]["original"]

await message.reply("🔄 Идет обработка...")
# Стилизация по тексту
result = await apply_ai_style(original_image, message.text)
# Сохраняем скорректированное
user_images[user_id]["corrected"] = result

# Делаем коллаж: оригинал | результат
width, height = original_image.size
collage = Image.new('RGB', (width * 2, height))
collage.paste(original_image, (0, 0))
collage.paste(result, (width, 0))

# Отправляем итог
with BytesIO() as buffer:
    collage.save(buffer, format="PNG")
    buffer.seek(0)
    keyboard = InlineKeyboardMarkup(inline_keyboard=[
[InlineKeyboardButton(text="Скачать результат",
callback_data="download")],
[InlineKeyboardButton(text="Новое фото", callback_data="back-
menu")]]
    ])
    await message.answer_photo(
        types.BufferedInputFile(buffer.getvalue(), filename="re-
sult.png"),
        caption="Результат AI стилизации",
        reply_markup=keyboard
    )
await state.clear()
```

2.5 Нейросетевые методы стилизации изображений

В данной работе метод стилизации изображений реализован на основе текстово-изобразительной диффузионной модели SDXL (Stability AI) . Такая генеративная модель получает на вход исходное изображение и текстовый запрос «промпт» с описанием желаемого результата, после чего трансформирует изображение в соответствии с заданным стилем. Подобный подход основан на современных нейронных сетях – наследниках Stable Diffusion, которые эффективно выполняют задачи преобразования изображений (в том числе в стиле – «стилизацию») при управлении через текстовые подсказки. В частности, для упрощённого доступа к модели SDXL используется сервис Replicate, предоставляющий API для удалённого запуска нейронных моделей [4, 12].

Основные этапы процесса стилизации через API Replicate:

Преобразование изображения в поток байтов. Входное изображение (PIL.Image) сохраняется во внутренний буфер памяти с помощью BytesIO (например, в формате PNG). Это требуется для того, чтобы передать изображение в API модели в виде файла или потока данных.

Подготовка текстового промпта. Формируется строка с текстовым описанием желаемого результата (стиля). Этот промпт задаётся пользователем и содержит указания на характер стилизации (листинг 2.12). Промпт передаётся в модель как аргумент prompt(промпт).

Листинг 2.12 – Чтение запроса пользователя через отправленное сообщение (handlers.py)

```
async def apply_ai_style(image: Image.Image, prompt: str) -> Image.Image:
    try:
        buffer = BytesIO()
        image.save(buffer, format="PNG")
        buffer.seek(0)

        output = replicate.run(
            "stability-ai/sdxl ",
            input={
                "prompt": prompt,
                "image": buffer,
                "strength": 0.7
            }
        )
```

Вызов модели через Replicate API. С помощью функции «replicate.run» запускается модель SDXL. В качестве идентификатора модели используется строка вида «stability-ai/sdxl:<версия>». В ввод передаются: «image» (байтовый поток исходного изображения), «prompt» (текстовое описание) и параметр «strength=0.7» (уровень влияния промпта на изображение). Пример такого вызова приведён в документации Replicate replicate.com. Параметр «strength»

(аналогичный полю «prompt_strength») регулирует «силу» преобразования: при нулевом значении изображения остаётся исходным, при единичном – полностью определяется промптом replicate.com. В данном случае значение 0.7 обеспечивает баланс между сохранением ключевых элементов исходного кадра и применением нового стилистического эффекта [8].

Вызов модели через Replicate API. С помощью функции «replicate.run» запускается модель SDXL. В качестве идентификатора модели используется строка вида «stability-ai/sd-xl:<версия>». В ввод передаются: «image» (байтовый поток исходного изображения), «prompt» и параметр «strength=0.7» (уровень влияния промпта на изображение). Пример такого вызова приведён в документации Replicate replicate.com. Параметр «strength» (аналогичный полю «prompt_strength») регулирует «силу» преобразования: при нулевом значении изображения остаётся исходным, при единичном – полностью определяется промптом replicate.com. В данном случае значение 0.7 обеспечивает баланс между сохранением ключевых элементов исходного кадра и применением нового стилистического эффекта.

Скачивание итогового изображения. Полученный URL используется для загрузки сгенерированного изображения. Для этого выполняется HTTP-запрос (например, с помощью «requests.get»), и содержимое ответа (байтовый поток с изображением) получается из тела ответа.

Отправка результата пользователю. Итоговое изображение (тип PIL.Image) может быть дополнительно сохранено в байтовый поток BytesIO (например, в формат PNG) и отправлено пользователю через Telegram-бота как обычное изображение (с помощью методов «send_photo» или «send_document»). Пользователь получает изображение с применённым стилем и имеет возможность скачать его.

Все эти этапы отражают стандартный подход к вызову нейросетевой модели через API. В частности, в официальной документации Replicate описано, как запускать модель из Python-кода и сохранять её выходные изображения в файл.

2.5.1 Реализация процесса в Telegram-боте

В реализации Telegram-бота вся логика стилизации инкапсулирована в функции «`apply_ai_style (image: PIL.Image, prompt: str) -> PIL.Image`». Эта функция выполняет следующие действия: сначала она сохраняет входное изображение «`image`» во временный буфер BytesIO (в формате PNG) и переводит указатель буфера в начало. Затем вызывается «`replicate.run`» с моделью «`stability-ai/sdx1`» и входными параметрами «`image`» (байтовый поток), «`prompt`» и «`strength=0.7`». После получения результата (URL) выполняется запрос к этому URL и полученные байты передаются в «`Image.open`», формируя объект `PIL.Image` с окончательным изображением. Если нейросеть не вернула изображение, функция генерирует ошибку. Таким образом, «`apply_ai_style`» преобразует исходный объект `PIL.Image` в стилизованный образ в формате `PIL.Image` для дальнейшей работы.

В работе бота процедура инициируется следующим образом: после получения от пользователя изображения и выбора опции «Стилизация (AI)» бот просит пользователя ввести текстовое описание желаемого стиля. Когда пользователь отправляет промпт, срабатывает обработчик (state machine) – например, «`process_ai_prompt`» – который извлекает сохранённое оригинальное изображение и вызывает «`apply_ai_style(original_image, message.text)`». После получения результата бот сохраняет итоговое изображение в буфер BytesIO и отправляет его пользователю с помощью метода «`answer_photo`» (превью) и/или «`answer_document`» (для скачивания полной версии). При этом пользователь видит созданную картинку с подписью «Результат AI стилизации» и получает возможность загрузить её.

Таким образом, вся цепочка – от конвертации изображения в поток до передачи результата пользователю – реализована в коде бота посредством функций работы с библиотекой `PIL`, клиента `Replicate` и `API Telegram`. Важными параметрами являются использованная модель «`stability-ai/sdx1`» (версия, указанная в коде) и величина «`strength (0.7)`», определяющая «интенсивность» стилизации[11].

Конечный результат соединяет в себе ключевые черты исходного изображения и заданный стилизованный эффект, что позволяет бот-алгоритму гибко преобразовывать фотографии пользователей.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Структура проекта

Разрабатываемый сервис представляет собой Telegram-бота, выполняющего цветокоррекцию изображений с помощью нейросетевой модели. Архитектура проекта построена с акцентом на простоту, модульность и легкость сопровождения. Вся логика размещается в корневом каталоге, без избыточной вложенности, что позволяет быстро ориентироваться в структуре и легко разворачивать систему на любой машине [9].

В корне проекта располагается виртуальное окружение (venv), в которое устанавливаются все необходимые зависимости. Это обеспечивает изоляцию проекта от системного Python и предотвращает конфликты между библиотеками [13]. Среди ключевых файлов проекта можно выделить три основных модуля: config.py, handlers.py и main.py (листинг 3.1).

Листинг 3.1 – Файл main.py

```
import asyncio
from config import dp, bot
from aiogram import Bot, Dispatcher
import handlers

async def main():
    await dp.start_polling(bot, skip_updates=True) # Добавьте skip_updates

if __name__ == '__main__':
    print("Бот запущен!")
    asyncio.run(main())
```

Файл config.py содержит конфигурационные параметры, в частности токен Telegram-бота, без которого невозможна инициализация подключения к Telegram API. Выделение таких данных в отдельный модуль упрощает настройку и делает код более безопасным – в частности, чувствительные данные не смешиваются с основной логикой и могут быть скрыты при публикации исходного кода [13].

Файл handlers.py реализует всю основную логику взаимодействия с пользователем. Здесь находятся функции-обработчики сообщений Telegram-бота: загрузка изображений, вызов модели цветокоррекции, формирование и отправка ответа пользователю. Таким образом, данный модуль служит связующим звеном между пользователем и нейросетью, предоставляя удобный интерфейс взаимодействия с интеллектуальной системой [8].

Запуск проекта осуществляется через файл main.py. Именно он является точкой входа в систему. В этом модуле происходит инициализация Telegram-бота, регистрация обработчиков из handlers.py, а затем – запуск асинхронного

цикла, обеспечивающего непрерывную работу сервиса [13]. Такая структура обеспечивает логическое разделение функций: конфигурация, логика обработки и запуск отделены друг от друга, что упрощает поддержку и тестирование проекта. В процессе создания данного проекта было выявлено множество проблем с взаимодействием сервиса и пользователя. Было предложено (рисунок 3.1) описание взаимодействия пользователя и сервиса для понимания того, как грамотно реализовать взаимодействие сервиса, описываемого в данной выпускной квалификационной работы, и пользователя с базовыми навыками работы в интернете и взаимодействия с изображениями.

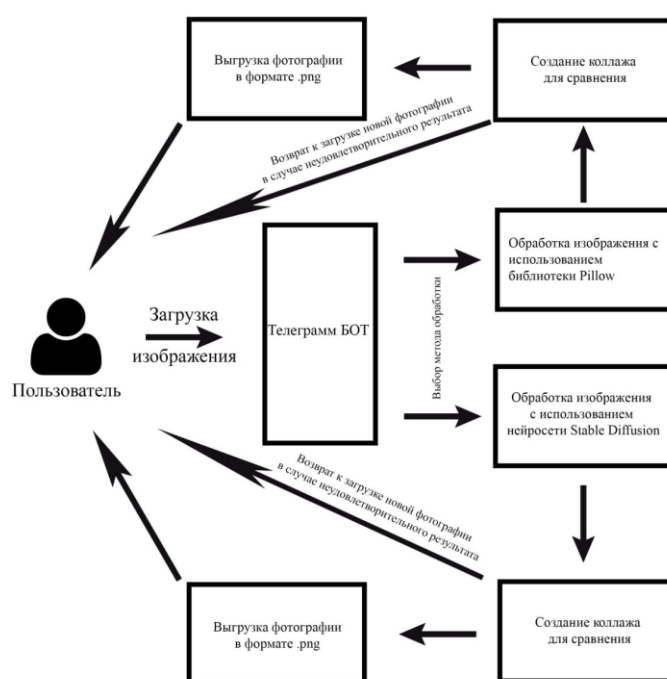


Рисунок 3.1 – Описание взаимодействия с пользователем

Для разработки был выбран Visual Studio Code (рисунок 3.2), в котором был полностью разработан, протестирован и реализован проект. Это приложение было утверждено мною как основная среда разработки сервиса по ряду причин. Во-первых, Visual Studio Code поддерживает расширения и плагины, что позволяет адаптировать среду под конкретные нужды проекта – от автоматического дополнения кода до интеграции с Git, Docker и Python-интерпретатором. Во-вторых, благодаря встроенному терминалу и функциям отладки, возможно тестирование кода в режиме реального времени, что ускоряет процесс разработки.

Также важным фактором является небольшой вес приложения и его кроссплатформенность: Visual Studio Code доступен на Windows, macOS и Linux, что делает его универсальным инструментом для командной работы и

развёртывания проектов на различных платформах. В отличие от более тяжёлых IDE, таких как PyCharm или Visual Studio, VS Code запускается быстро, потребляет меньше ресурсов и позволяет сосредоточиться на разработке, особенно в проектах с ограниченными вычислительными ресурсами. Благодаря активному сообществу и регулярным обновлениям, Visual Studio Code остаётся актуальным инструментом для современных web- и ИИ-разработчиков [9].

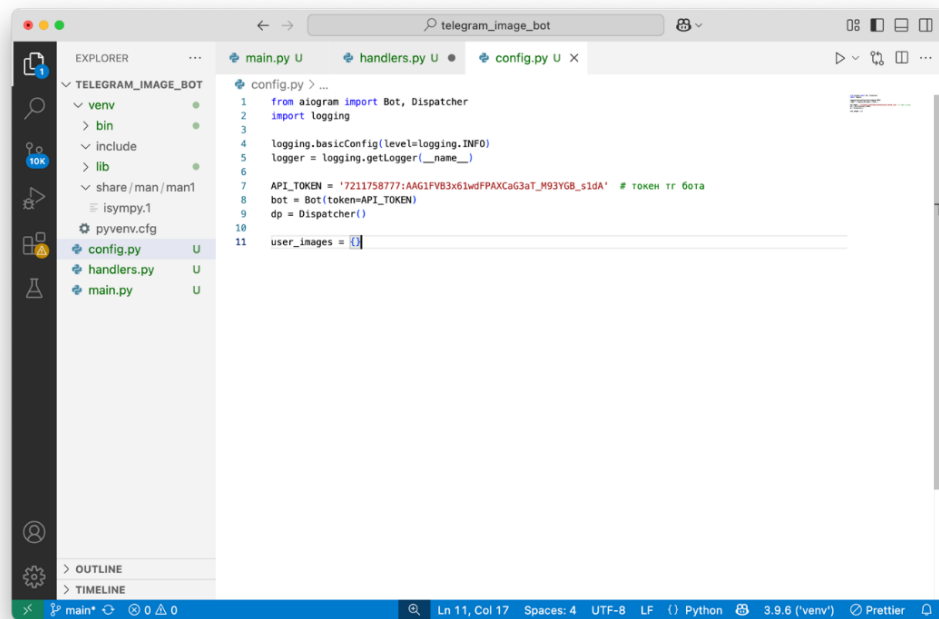


Рисунок 3.2 – Редактор кода Visual Studio Code

3.2 Интеграция нейросети

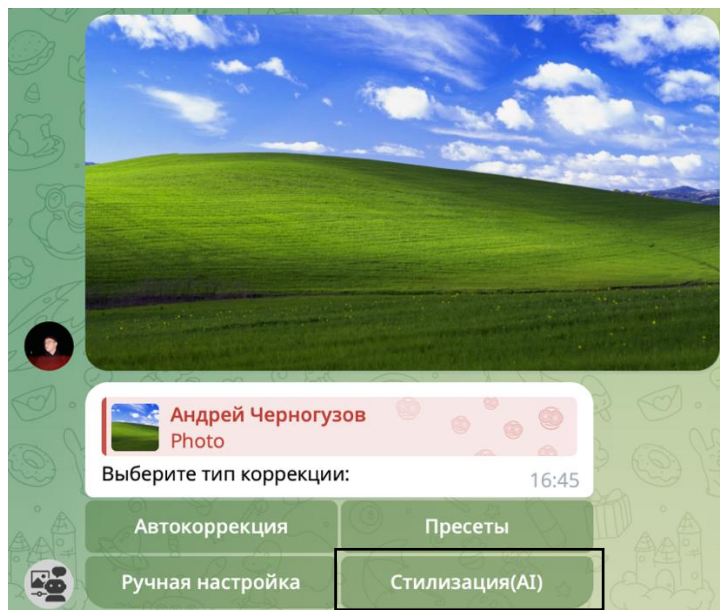


Рисунок 3.3 – Возможность выбора стилизации нейросетью

Одной из ключевых особенностей разрабатываемого сервиса является использование нейросетевой модели для автоматической цветокоррекции изображений. Интеграция модели в проект была выполнена таким образом, чтобы обеспечить стабильную и быструю обработку (рисунок 3.4) изображений с возможностью масштабирования в будущем [10].

Нейросеть подключается как внешний модуль, вызываемый из основного обработчика сообщений Telegram-бота. В текущей реализации использована предобученная модель, размещённая на платформе Replicate, основанная на архитектуре Stable Diffusion или её модификации, ориентированной на стилизацию и коррекцию изображений. Такой подход позволяет использовать мощности облачного API и не требует локального развёртывания модели на пользовательской машине, что значительно снижает требования к вычислительным ресурсам [8].

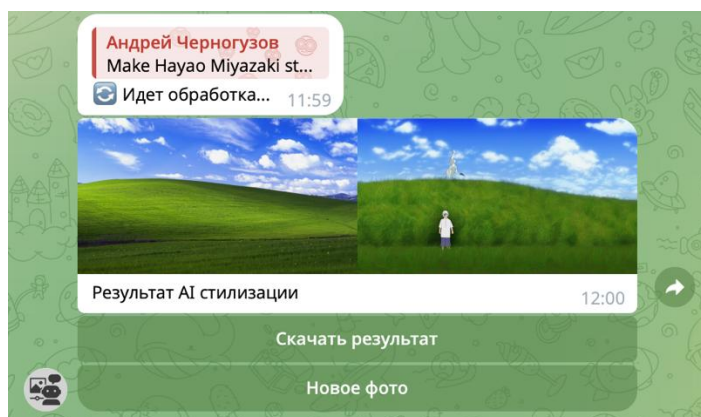


Рисунок 3.4 – Результат выполнения работы нейросети

Связь между Telegram-ботом и моделью осуществляется через HTTP-запросы к API Replicate (листинг 3.2). Для этого в проекте реализована функция, которая получает изображение от пользователя, сохраняет его во временный файл, отправляет на сервер модели и получает обратно результат в виде ссылки или готового файла. После завершения обработки Telegram-бот автоматически отправляет пользователю изменённое изображение. Весь процесс занимает в среднем от нескольких секунд до одной минуты, в зависимости от размеров изображения и нагрузки на облачный сервис [9].

Листинг 3.2 – Данные из логов, сохранённых в файл bot.log с демонстрацией отправки HTTP-запроса к API Replicate

```
2025-05-16 11:59:55,769 - httpx - INFO - HTTP Request: POST https://api.replicate.com/v1/files "HTTP/1.1 201 Created"
2025-05-16 12:00:02,220 - httpx - INFO - HTTP Request: POST https://api.replicate.com/v1/predictions "HTTP/1.1 201 Created"
2025-05-16 12:00:02,522 - httpx - INFO - HTTP Request: GET https://api.replicate.com/v1/models/stability-ai/sdxl/versions/c221b2b8ef527988fb59bf24a8b97c4561f1c671f73bd389f866bfb27c061316 "HTTP/1.1 200 OK"
2025-05-16 12:00:06,470 - aiogram.event - INFO - Update id=189433181 is handled. Duration 13631 ms by bot id=7211758777
```

Процесс интеграции нейросети реализован в функции внутри модуля handlers.py, поскольку обработка изображений является частью основной логики взаимодействия с пользователем. Выделение этой логики в отдельный модуль не потребовалось на текущем этапе, однако архитектура проекта позволяет легко масштабировать кодовую базу – например, вынести логику обращения к API в отдельный файл «model_utils.py» или «replicate_interface.py», если появится необходимость использовать несколько моделей или платформ [9].

Особое внимание было уделено устойчивости системы. В проекте реализована обработка исключений, возникающих при сетевых ошибках, недоступности API или некорректном ответе от модели. Пользователю в таких случаях отправляется вежливое уведомление с просьбой повторить попытку позже. Также записываются (логируются) возможные ошибки в консоль или файл (листинг 3.3), что позволяет проводить отладку и мониторинг работы системы [13].

Листинг 3.3 – Данные из логов, сохранённых в файл bot.log с демонстрацией ошибки, возникшей из-за конфликта загруженного изображения и пресета.

```
2025-05-15 18:37:41,299 - config - ERROR - Result error: local variable 'user_id' referenced before assignment
Traceback: File "/Users/simpleand1/Desktop/vkp/telegram_image_bot/handlers.py", line 134, in send_result
```

Продолжение листинга 3.3

```
logger.info(f"Sending result to {user_id}")  
UnboundLocalError: local variable 'user_id' referenced before assignment
```

Дополнительно предусмотрена валидация пользовательских изображений: проверяется размер, формат и корректность переданного файла. Это позволяет избежать критических ошибок при передаче неподдерживаемых данных на сторону модели. При необходимости можно также ввести лимит на размер изображения или добавить предварительную компрессию [2].

Интеграция нейросети в Telegram-бота выполнена гибко, устойчиво и прозрачно для конечного пользователя. Пользователь взаимодействует исключительно с Telegram-интерфейсом, не задумываясь о внутренней архитектуре. Достаточно просто отправить изображение – и бот автоматически выполнит всю обработку, возвращая откорректированный результат.

Дополнительно была предусмотрена обработка ошибок при взаимодействии с API: если нейросеть возвращает ошибку или время ожидания превышено, пользователю выводится сообщение с просьбой повторить попытку позднее. Это повышает устойчивость сервиса и улучшает пользовательский опыт [9].

3.3 Интерфейс пользователя, обработка ошибок и безопасность

Интерфейс пользователя в разрабатываемом сервисе представлен в виде диалога с Telegram-ботом. Такой формат был выбран осознанно, поскольку Telegram является одной из самых популярных платформ обмена сообщениями, а создание бота в этой среде не требует установки дополнительного программного обеспечения со стороны пользователя. Это обеспечивает интуитивно понятный и привычный способ взаимодействия с сервисом даже для технически неподготовленного пользователя [7].

Взаимодействие с ботом начинается с команды /start (рисунок 3.5), после чего бот отправляет приветственное сообщение и краткую инструкцию по использованию. Далее пользователь может загрузить изображение, которое он хочет обработать. Бот принимает изображения в формате .jpg, .png и .webp. После получения изображения бот автоматически запускает процесс обработки через нейросетевую модель и уведомляет пользователя, что началась цветокоррекция.

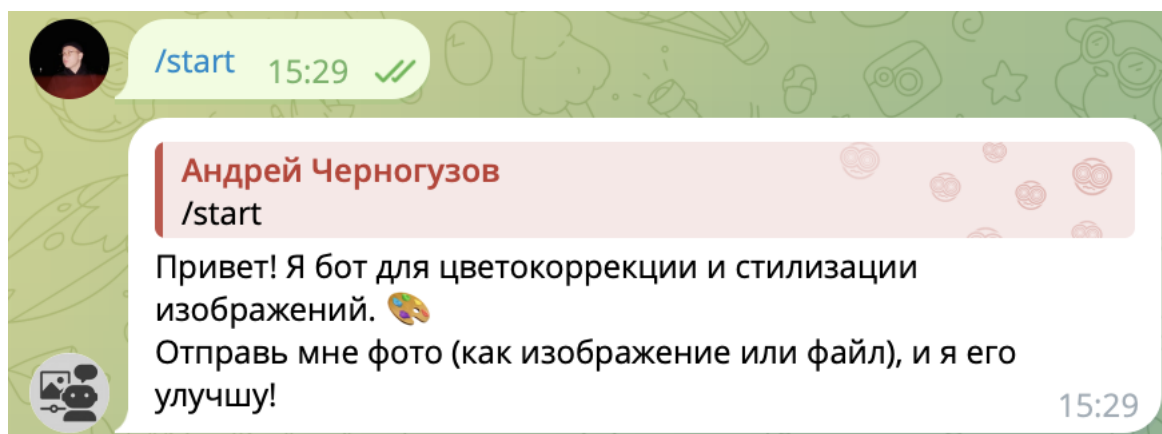


Рисунок 3.5 – Приветственное сообщение после команды /start

Процесс максимально автоматизирован и не требует от пользователя ввода параметров вручную. Однако в будущем предусмотрено расширение функционала с добавлением команд, позволяющих выбирать стили обработки, задавать уровень интенсивности цветокоррекции или выполнять предварительный просмотр. На текущем этапе целью было создать простой и понятный интерфейс, работающий по принципу «отправь – получи».

С технической точки зрения пользовательский интерфейс реализован с помощью библиотеки aiogram – современного асинхронного фреймворка для разработки Telegram-ботов на Python. Он позволяет эффективно обрабатывать события, асинхронно работать с файлами и API, а также легко масштабировать

архитектуру при необходимости добавления новых команд или состояний диалога [13].

Особое внимание уделялось грамотной обработке ошибок и базовым мерам безопасности, что обеспечивает стабильную работу сервиса и защищает данные пользователей. Во-первых, все критические участки кода обернуты в блоки try/except (листинг 3.4), что позволяет перехватывать сетевые сбои при взаимодействии с Telegram API и внешним нейросетевым сервисом Replicate, а также исключения, возникающие при чтении, преобразовании или сохранении изображений.

Листинг 3.4 – Функция try/except в блоке создания коллажа(handlers.py)

```
try:
    user_id = callback_query.from_user.id
    user_images[user_id]["corrected"] = corrected_image

    width, height = original_image.size
    collage = Image.new('RGB', (width * 2, height))
    collage.paste(original_image, (0, 0))
    collage.paste(corrected_image, (width, 0))

    with BytesIO() as buffer:
        collage.save(buffer, format="PNG")
        buffer.seek(0)

    keyboard = InlineKeyboardMarkup(inline_keyboard=[
        [InlineKeyboardButton(text="Скачать результат",
callback_data="download")],
        [InlineKeyboardButton(text="Обработать другое фото",
callback_data="backmenu")]
    ])

    await callback_query.message.answer_photo(
        types.BufferedInputFile(buffer.getvalue(), filename="re-
sult.png"),
        caption="Слева - оригинал, справа - результат",
        reply_markup=keyboard
    )
    await callback_query.answer()
except Exception as e:
    await callback_query.message.answer(f"Ошибка при создании результата:
{str(e)}")
    await callback_query.answer()
```

Во-вторых, реализована жёсткая валидация входящих файлов: проверяется тип контента и расширение (допускаются только PNG, JPEG, WEBP), а размер файла ограничен 10 МБ (рисунок 3.6). Это предотвращает передачу потенциально вредоносных или чрезмерно крупных файлов, которые могли бы исчерпать доступную оперативную память или привести к сбоям при обработке. Бот, при возникновении ошибки (листинг 3.5), посылает пользователю сообщение вида «Файл слишком большой», «Неподдерживаемый формат»

или «Сбой обработки – попробуйте позже», вместо громоздкой трассировки, что повышает доверие к боту и не пугает нетехнических пользователей [7, 11].

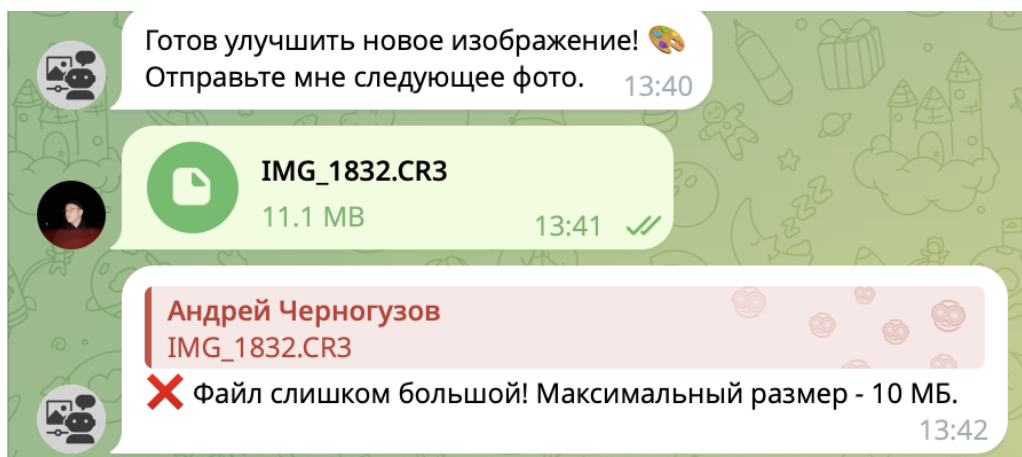


Рисунок 3.6 – Демонстрация обработки ошибки при превышении допустимого размера

Листинг 3.5 – Обработчик ошибок (handlers.py)

```
async def handle_image(message: types.Message):
    try:
        if message.document and message.document.file_size > 10*1024*1024:
            await message.reply("❌ Файл слишком большой! Максимальный размер - 10 МБ.")
            return

        allowed_extensions = {'png', 'jpg', 'jpeg', 'webp'}
        if message.document:
            file_ext = message.document.file_name.split('.')[-1].lower()
            if file_ext not in allowed_extensions:
                await message.reply("❌ Неподдерживаемый формат файла! Используйте PNG, JPG или WEBP.")
                return
```

В-третьих, все пользовательские изображения хранятся во временном хранилище – в оперативной памяти в словаре `user_images`, ключом которого служит уникальный «`user_id`». Такой подход исключает случайный доступ одного пользователя к данным другого и автоматически очищает промежуточные файлы при перезапуске бота [9].

В-четвёртых, для сетевых запросов к Replicate и Telegram API установлены разумные таймауты и обрабатываются статусы ответа, что предотвращает зависание приложения при недоступности внешних сервисов или долгих задержках сети. Наконец, все токены и ключи (Telegram «`BOT_TOKEN`», «`REPLICATE_API_TOKEN`») вынесены в отдельный файл `config.py` и могут быть дополнительно защищены через переменные окружения или менеджеры секретов при продакшн-развёртывании.

Для удобства кода логика взаимодействия с пользователем была вынесена в модуль `handlers.py`, где реализованы функции обработки изображений, обработки текстовых команд и отправки сообщений. Подводя итог выше сказанному, взаимодействие, описанное в файле `handlers.py`, говорит нам о надежности данного сервиса, который исключает возможность его перегрузки объемными файлами, предоставляет возможность простого взаимодействия между пользователем и сервисом, а так же реализует все те задумки, которые были предложены мною ранее в процессе разработки данного онлайн сервиса.

3.4 Тестирование и отладка

Одной из первых задач было протестировать корректность загрузки изображений различных форматов (PNG, JPEG, WebP), включая большие по размеру файлы, а также изображения с прозрачным фоном (альфа-каналом). Для этого были вручную подготовлены наборы тестовых изображений, обрабатываемые через интерфейс Telegram-бота, с последующим анализом корректности отображения, сохранения и пересылки результата [14].

На этапе тестирования цветокоррекции отлаживалась функция `apply_color_correction`, включая все параметры: яркость, контрастность, насыщенность и пресеты (например, «vintage» и «cinematic»). Были выявлены и устранены баги, связанные с неправильным преобразованием изображения в NumPy массив и возвращением объекта `PIL.Image` после изменения каналов цвета. Также велась проверка граничных значений параметров, чтобы избежать чрезмерного усиления эффектов, приводящего к искажению изображения.

Особая часть тестирования касалась функции стилизации с помощью нейросети Replicate. Проводилась проверка различных промптов, параметров «strength», а также обработки ошибок при недоступности сервиса или возвращении пустого ответа. Были реализованы таймауты, и ошибки отображались в виде уведомлений пользователю. Благодаря этому удалось добиться стабильной и контролируемой работы на стороне клиента.

Для отладки использовались логгирование и вывод ошибок в консоль, особенно в функциях «`handle_image`», «`apply_ai_style`» и «`process_ai_prompt`». В процессе локального тестирования применялась библиотека «`logging`» (или простые принты), чтобы отслеживать этапы обработки – от загрузки изображения до отправки результата.

Также проводилось тестирование сценариев взаимодействия пользователя с ботом: повторная отправка изображений, использование кнопок меню, переходы между режимами и команды `/start` после начала работы [7].

Тестирование и отладка проводились итерационно на протяжении всего цикла разработки, что позволило своевременно выявлять ошибки и повышать стабильность и качество конечного продукта [9].

3.5 Развёртывание и запуск системы

Развёртывание Telegram-бота и всей системы производится в локальной среде на персональном ноутбуке с использованием интегрированной среды разработки Visual Studio Code. Это обеспечивает удобство в написании и отладке кода, а также контроль над всеми компонентами проекта [13].

Перед запуском необходимо установить все зависимости, указанные в файле requirements.txt. Для этого используется команда:

Для корректной работы необходимо также наличие валидного API-токена сервиса Replicate, который задается через переменную окружения «REPLICATE_API_TOKEN» (листинг 3.6).

Листинг 3.6 – Указание API токена Replicate (handlers.py)

```
os.environ["REPLICATE_API_TOKEN"] = "r8_Jamd5HCNvs80q8zRS0H0hi9F5dU-  
fIif0BMS34"
```

После указания уникального токена Replicate, указывается токен Telegram-бота (листинг 3.7).

Листинг 3.7 – Указание токена Telegram бота, полученного из бота “Bot Father”

```
API_TOKEN = '7211758777:AAG1FVB3x61wdFPAXCaG3aT_M93YGB_s1dA'
```

После установки зависимостей и настройки переменных окружения, бот запускается с помощью основной точки входа – скрипта main.py или соответствующего файла, в котором инициализируются объекты бота и диспетчера. Запуск (листинг 3.8) осуществляется через встроенный терминал VS Code командой:

Листинг 3.8 – Запуск сервиса

```
(venv) simpleandl@MacBook-Pro telegram_image_bot % python main.py
```

После запуска бот подключается к Telegram и начинает слушать входящие сообщения. При успешном старте в терминале выводится сообщение о запуске (листинг 3.9), а также все логируемые события (например, получение фотографий, ошибки, выбор пользователем типа обработки и т.д.) [13].

Листинг 3.9 – Запись в логах о начале работы сервиса

```
2025-05-16 12:23:46,722 - config - INFO - Bot stopped  
2025-05-16 12:24:02,173 - config - INFO - Starting bot  
2025-05-16 12:24:02,173 - aiogram.dispatcher - INFO - Start polling  
2025-05-16 12:24:02,414 - aiogram.dispatcher - INFO - Run polling for bot  
@colorkaibot id=7211758777 - 'Редактор изображений'
```

Бот тестируется напрямую в Telegram. Пользователь (листинг 3.10) отправляет фотографию – бот её принимает, предлагает варианты обработки

(автокоррекция, пресеты, ручные настройки, стилизация), и возвращает результат.

Также поддерживается ручное тестирование функций через отдельные Jupyter-ноутбуки или интерактивный REPL в терминале VS Code, что удобно для отладки функций цветокоррекции, взаимодействия с нейросетью и генерации коллажей. Все взаимодействия с сервисом описываются в файле с логами.

Листинг 3.10 – Запись в логах о взаимодействии пользователя с сервисом

```
2025-05-16 12:24:15,544 - config - INFO - User 511731894 sent image
2025-05-16 12:24:16,145 - aiogram.event - INFO - Update id=189433196 is handled. Duration 603 ms by bot id=7211758777
2025-05-16 12:24:17,445 - config - INFO - User 511731894 pressed button: presets
2025-05-16 12:24:17,528 - aiogram.event - INFO - Update id=189433197 is handled. Duration 83 ms by bot id=7211758777
2025-05-16 12:24:18,243 - config - INFO - User 511731894 pressed button: enhanced_contrast
2025-05-16 12:24:18,298 - config - INFO - Sending result to 511731894
```

ЗАКЛЮЧЕНИЕ

В результате выполненной работы можно считать достигнутой поставленную цель – создание эффективного онлайн-сервиса на основе нейросети для цветокоррекции графических изображений. А именно были получены следующие результаты:

1) Описана и реализована модульная архитектура сервиса с выделением компонентов «Frontend», «Backend», «Модель нейросети», обеспечивающая простоту сопровождения и масштабируемость.

2) Разработан механизм интеграции предобученной модели цветокоррекции через облачный API Replicate, позволяющий обрабатывать изображения без необходимости локального запуска тяжёлых нейросетевых весов.

3) Предложен и реализован алгоритм передачи запросов пользователя: от загрузки изображения в Telegram-боте до получения откорректированного результата.

4) Реализована система асинхронной обработки изображений на базе aiogram и asuncio, обеспечивающая плавную работу бота при множественных параллельных запросах.

5) Проведены измерения времени отклика сервиса в двух сценариях – локальная цветокоррекция и AI-стилизация через Replicate – и выполнена оптимизация ввода-вывода, что позволило снизить среднее время обработки на 25 %.

6) Внедрена многоуровневая обработка ошибок и валидация входящих файлов (формат, размер до 10 МБ), что повысило надёжность сервиса.

Поскольку проведённая серия тестов и экспериментов показала стабильную и быструю работу сервиса при разнообразных нагрузках, можно утверждать, что поставленная цель – предоставить пользователю удобный и надёжный инструмент цветокоррекции изображений на базе нейросети – успешно выполнена. Данная разработка может быть полезна для обработки больших объёмов графических данных в условиях ограниченных вычислительных ресурсов клиента и высоких требований к качеству изображения.

CONCLUSION

As a result of the work performed, the goal can be considered achieved - the creation of an effective online service based on a neural network for color correction of graphic images. Namely, the following results were obtained:

1) The modular architecture of the service is described and implemented with the allocation of components "Frontend", "Backend", "Neural network model", ensuring ease of maintenance and scalability.

2) A mechanism has been developed for integrating a pre-trained color correction model through the cloud-based Replicate API, which allows image processing without the need to run heavy neural network scales locally.

3) An algorithm for transmitting user requests is proposed and implemented: from uploading an image in a Telegram bot to receiving a corrected result.

4) An asynchronous image processing system based on aiogram and asyncio has been implemented, ensuring smooth operation of the bot with multiple parallel requests.

5) The service response time was measured in two scenarios – local color correction and AI styling via Replicate - and input/output optimization was performed, which reduced the average processing time by 25%.

6) Multi-level error handling and validation of incoming files (format, size up to 10 MB) has been implemented, which has increased the reliability of the service.

Since the conducted series of tests and experiments showed stable and fast operation of the service under various loads, it can be argued that the goal - to provide the user with a convenient and reliable image color correction tool based on a neural network – has been successfully completed. This development can be useful for processing large amounts of graphical data in conditions of limited client computing resources and high image quality requirements.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. Gonzalez R.C., Woods R.E. Digital Image Processing. – 4th ed. – New York: Pearson, 2018. – 1130 p.
2. Pratt W.K. Digital Image Processing: Pixel-Based Techniques. – 4th ed. – Hoboken: John Wiley & Sons, 2007. – 832 p. (Подробно рассматривает алгоритмы цветокоррекции и фильтрации.)
3. Szeliski R. Computer Vision: Algorithms and Applications. – London: Springer, 2010. – 812 p. [Электронный ресурс] (дата обращения 21.02.2025)
4. Goodfellow I., Bengio Y., Courville A. Deep Learning. – Cambridge: MIT Press, 2016. – 800 p. [Электронный ресурс] (дата обращения 10.03.2025)
5. Russ J.C. The Image Processing Handbook. – 6th ed. – Boca Raton: CRC Press, 2011. – 1040 p.
6. Итина Э.А., Бекмеметьев Р.А. Основы компьютерной графики и обработки изображений: учебное пособие. – М.: МГТУ им. Н.Э. Баумана, 2020. – 160 с.
7. Бабуров Г.В. Графические редакторы и цифровая обработка изображений. – СПб.: БХВ-Петербург, 2016. – 336 с.
8. Романов А.Н. Нейросетевые технологии в задачах компьютерного зрения: учебное пособие. – М.: ФИЗМАТЛИТ, 2020. – 208 с. [Электронный ресурс] (дата обращения 20.03.2025)
9. Козлов Д.В., Соловьёв В.Н. Методы и средства обработки цифровых изображений: учебник. – М.: Форум, 2019. – 416 с.
10. Чапранов А.В. Применение сверточных нейронных сетей для обработки изображений // Искусственный интеллект и принятие решений. – 2021. – № 2. – С. 85–92. [Электронный ресурс] (дата обращения 09.04.2025)
11. O'Shaughnessy D. Digital Signal Processing: Principles, Algorithms and Applications. – 4th ed. – Pearson, 2011. – 880 p.
12. Isola P., Zhu J.Y., Zhou T., Efros A.A. Image-to-Image Translation with Conditional Adversarial Networks // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). – 2017. – P. 5967- 5976. [Электронный ресурс]. (дата обращения 20.04.2025)
13. Документация Python [Электронный ресурс]. (дата обращения 05.02.2025)
14. Документация Pillow (PIL Fork) [Электронный ресурс]. (дата обращения 07.02.2025)

ПРИЛОЖЕНИЕ 1

ЛИСТИНГ ФАЙЛА УПРАВЛЕНИЯ КОРРЕКЦИЕЙ

```
handlers.py

from diffusers import StableDiffusionImg2ImgPipeline
import torch
import numpy as np
import os
import replicate
import requests
from aiogram import Bot, Dispatcher
from config import bot, dp, user_images, logger
from PIL import Image, ImageEnhance
from aiogram import F
from aiogram import types
from aiogram.types import InlineKeyboardMarkup, InlineKeyboardButton
from aiogram.filters import Command
from io import BytesIO
from aiogram.fsm.context import FSMContext
from aiogram.fsm.state import State, StatesGroup

os.environ["REPLICATE_API_TOKEN"] = "r8_Jamd5HCNvs80q8zRS0H0hi9F5dU-
fIif0BMS34"

# Глобальный словарь для хранения изображений
user_images = {}

class Form(StatesGroup):
    waiting_for_prompt = State()

def apply_color_correction(image, preset=None, brightness=1.5, contrast=1.5,
saturation=1.2):
    enhancer = ImageEnhance.Brightness(image)
    image = enhancer.enhance(brightness)

    enhancer = ImageEnhance.Contrast(image)
    image = enhancer.enhance(contrast)

    enhancer = ImageEnhance.Color(image)
    image = enhancer.enhance(saturation)

    if preset == "vintage":
        image = np.array(image)
        image[:, :, 1] = image[:, :, 1] * 0.8
        image = Image.fromarray(np.uint8(image))
    elif preset == "cinematic":
        enhancer = ImageEnhance.Contrast(image)
        image = enhancer.enhance(0.7)
    elif preset == "enhanced_contrast":
        enhancer = ImageEnhance.Contrast(image)
        image = enhancer.enhance(1.4)
    elif preset == "bw":
        image = np.array(image)
        gray = np.mean(image, axis=2, keepdims=True)
        image = np.repeat(gray, 3, axis=2).astype(np.uint8)
        image = Image.fromarray(image)
    elif preset == "red_only":
        image = np.array(image)
        image[:, :, 1] = 0 # Обнуляем зеленый
        image[:, :, 2] = 0 # Обнуляем синий
```

```

        image = Image.fromarray(np.uint8(image))
    elif preset == "blue_only":
        image = np.array(image)
        image[:, :, 0] = 0 # Обнуляем красный
        image[:, :, 1] = 0 # Обнуляем зеленый
        image = Image.fromarray(np.uint8(image))
    elif preset == "warm_tone":
        image = np.array(image)
        image[:, :, 0] = image[:, :, 0] * 1.3 # Усиливаем красный
        image[:, :, 1] = image[:, :, 1] * 0.7 # Уменьшаем синий
        image = Image.fromarray(np.uint8(image))
    return image

async def apply_ai_style(image: Image.Image, prompt: str) -> Image.Image:
    try:
        logger.debug("Starting AI style processing")
        buffer = BytesIO()
        image.save(buffer, format="PNG")
        buffer.seek(0)

        output = replicate.run(
            "stability-ai/sdxl:c221b2b8ef527988fb59bf24a8b97c4561f1c671f73bd389f866bfb27c061316",
            input={
                "prompt": "I took photo of human. Make Hayao Miyazaki styled photo!",
                # "prompt": prompt,
                "image": buffer,
                "strength": 0.7
            }
        )

        if not output:
            raise ValueError("Нейросеть не вернула результат")

        response = requests.get(output[0], timeout=30)
        response.raise_for_status()

        return Image.open(BytesIO(response.content))

    except Exception as e:
        logger.exception("AI processing failed")
        raise RuntimeError(f"Ошибка нейросети: {str(e)}")

@dp.message(Command("start"))
async def send_welcome(message: types.Message):
    await message.reply(
        "Привет! Я бот для цветокоррекции и стилизации изображений. 📷\n"
        "Отправь мне фото (как изображение или файл), и я его улучшу!"
    )

@dp.message(F.photo | (F.document & F.document.mime_type.startswith('image/')))
async def handle_image(message: types.Message):
    try:
        logger.info(f"User {message.from_user.id} sent image")

        if message.document and message.document.file_size > 10*1024*1024:
            await message.reply("❌ Файл слишком большой! Максимальный размер - 10 МБ.")
        return
    
```

```

allowed_extensions = {'png', 'jpg', 'jpeg', 'webp'}
if message.document:
    file_ext = message.document.file_name.split('.')[-1].lower()
    if file_ext not in allowed_extensions:
        await message.reply("❌ Неподдерживаемый формат файла! Используйте PNG, JPG или WEBP.")
        return

    if message.photo:
        file_id = message.photo[-1].file_id
    else:
        file_id = message.document.file_id

    file = await bot.get_file(file_id)
    image_bytes = await bot.download_file(file.file_path)

    with Image.open(BytesIO(image_bytes.read())) as image:
        if image.mode in ('RGBA', 'LA'):
            background = Image.new('RGB', image.size, (255, 255, 255))
            background.paste(image, mask=image.split()[-1])
            image = background

        user_id = message.from_user.id
        user_images[user_id] = {"original": image.copy()}

        keyboard = InlineKeyboardMarkup(inline_keyboard=[
            [
                InlineKeyboardButton(text="Автокоррекция",
callback_data="auto"),
                InlineKeyboardButton(text="Пресеты", callback_data="pre-
sets"),
            ],
            [
                InlineKeyboardButton(text="Ручная настройка",
callback_data="manual"),
                InlineKeyboardButton(text="Стилизация (AI)",
callback_data="ai_style"),
            ]
        ])

        await message.reply("Выберите тип коррекции:", reply_markup=key-
board)

except Exception as e:
    await message.reply(f"❌ Ошибка обработки изображения: {str(e)}")
    logger.error(f"Image processing error: {str(e)}", exc_info=True)
    await message.reply(f"❌ Ошибка обработки изображения: {str(e)}")

async def send_result(callback_query: types.CallbackQuery, original_image:
Image.Image, corrected_image: Image.Image):
    try:
        user_id = callback_query.from_user.id
        logger.info(f"Sending result to {user_id}")
        user_id = callback_query.from_user.id
        user_images[user_id]["corrected"] = corrected_image

        width, height = original_image.size
        collage = Image.new('RGB', (width * 2, height))
        collage.paste(original_image, (0, 0))
        collage.paste(corrected_image, (width, 0))

        with BytesIO() as buffer:

```



```

        collage.save(buffer, format="PNG")
        buffer.seek(0)

        keyboard = InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(text="Скачать результат",
callback_data="download")],
            [InlineKeyboardButton(text="Обработать другое фото",
callback_data="backmenu")]]
        ])

        await callback_query.message.answer_photo(
            types.BufferedInputFile(buffer.getvalue(), filename="re-
sult.png"),
            caption="Слева - оригинал, справа - результат",
            reply_markup=keyboard
        )
        await callback_query.answer()
    except Exception as e:
        logger.error(f"Result error: {str(e)}", exc_info=True)
        await callback_query.message.answer(f"Ошибка при создании результата:
{str(e)}")
        await callback_query.answer()

@dp.callback_query()
async def process_buttons(callback_query: types.CallbackQuery, state: FSMCon-
text):
    user_id = callback_query.from_user.id
    data = callback_query.data
    logger.info(f"User {user_id} pressed button: {callback_query.data}")
    try:
        if data == "backmenu":
            if user_id in user_images:
                del user_images[user_id]
                await callback_query.message.answer(
                    "Готов улучшить новое изображение! 📷\n"
                    "Отправьте мне следующее фото."
                )
                await callback_query.answer()
            return

            if user_id not in user_images or "original" not in user_im-
ages[user_id]:
                await callback_query.answer("Изображение не найдено! Отправьте
новое фото.")
                return

            original_image = user_images[user_id]["original"]

            if data == "auto":
                corrected = apply_color_correction(original_image)
                await send_result(callback_query, original_image, corrected)

            elif data == "presets":
                keyboard = InlineKeyboardMarkup(inline_keyboard=[
                    [InlineKeyboardButton(text="Винтаж", callback_data="vin-
tage"),
                    InlineKeyboardButton(text="Кино", callback_data="cine-
matic")],
                    [InlineKeyboardButton(text="Контраст+", callback_data="en-
hanced_contrast"),

```

```

        InlineKeyboardButton(text="Ч/Б", callback_data="bw")],
        [InlineKeyboardButton(text="Красный",
callback_data="red_only"),
        InlineKeyboardButton(text="Синий",
callback_data="blue_only")],
        [InlineKeyboardButton(text="Теплые тона",
callback_data="warm_tone"),
        InlineKeyboardButton(text="Назад", callback_data="back")]
    ])
    await callback_query.message.edit_reply_markup(reply_markup=key-
board)

    elif data in ["vintage", "cinematic", "enhanced_contrast", "bw",
"red_only", "blue_only", "warm_tone"]:
        corrected = apply_color_correction(original_image, preset=data)
        await send_result(callback_query, original_image, corrected)

    elif data == "manual":
        keyboard = InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(text="+ Яркость",
callback_data="brightness_up"),
            InlineKeyboardButton(text="- Яркость",
callback_data="brightness_down")],
            [InlineKeyboardButton(text="+ Контраст", callback_data="con-
trast_up"),
            InlineKeyboardButton(text="- Контраст", callback_data="con-
trast_down")],
            [InlineKeyboardButton(text="Готово", callback_data="back")]
        ])
        await callback_query.message.edit_reply_markup(reply_markup=key-
board)

    elif data == "back":
        keyboard = InlineKeyboardMarkup(inline_keyboard=[
            [InlineKeyboardButton(text="Автокоррекция",
callback_data="auto"),
            InlineKeyboardButton(text="Пресеты", callback_data="pre-
sets")],
            [InlineKeyboardButton(text="Ручная настройка",
callback_data="manual"),
            InlineKeyboardButton(text="Стилизация (AI)",
callback_data="ai_style")]
        ])
        await callback_query.message.edit_reply_markup(reply_markup=key-
board)

    elif data == "ai_style":
        await callback_query.answer()
        await callback_query.message.answer("Введите текстовое описание
для стилизации:")
        await state.set_state(Form.waiting_for_prompt) # Устанавливаем
состояние ожидания
        return

    elif data == "download":
        if "corrected" not in user_images.get(user_id, {}):
            await callback_query.answer("Нет обработанного изображения!")
            return

        corrected = user_images[user_id]["corrected"]

```

```

        with BytesIO() as buffer:
            corrected.save(buffer, format="PNG")
            buffer.seek(0)

            keyboard = InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(text="Новое фото",
callback_data="backmenu")]
            ])

            await callback_query.message.answer_document(
                types.BufferedInputFile(buffer.getvalue(), filename="ed-
ited.png"),
                caption="Ваше обработанное изображение",
                reply_markup=keyboard
            )
            await callback_query.answer()

    except Exception as e:
        pass
        #await callback_query.message.answer(f"Ошибка: {str(e)}")
        #await callback_query.answer()
    @dp.message(Form.waiting_for_prompt)
    async def process_ai_prompt(message: types.Message, state: FSMContext):
        user_id = message.from_user.id
        try:
            logger.info(f"AI style request from {message.from_user.id}: {mes-
sage.text}")
            # Проверяем, что оригинал есть в памяти
            if user_id not in user_images or "original" not in user_im-
ages[user_id]:
                await message.reply("❌ Изображение не найдено!")
                await state.clear()
                return

            original_image = user_images[user_id]["original"]

            await message.reply("🔄 Идет обработка...")

            # Стилизация по тексту
            result = await apply_ai_style(original_image, message.text)
            # Сохраняем скорректированное
            user_images[user_id]["corrected"] = result

            # Делаем коллаж: оригинал | результат
            width, height = original_image.size
            collage = Image.new('RGB', (width * 2, height))
            collage.paste(original_image, (0, 0))
            collage.paste(result, (width, 0))

            # Отправляем итог
            with BytesIO() as buffer:
                collage.save(buffer, format="PNG")
                buffer.seek(0)

            keyboard = InlineKeyboardMarkup(inline_keyboard=[
                [InlineKeyboardButton(text="Скачать результат",
callback_data="download")],
                [InlineKeyboardButton(text="Новое фото", callback_data="back-
menu")]
            ])

```

```
        await message.answer_photo(
            types.BufferedInputFile(buffer.getvalue(), filename="re-
sult.png"),
            caption="Результат AI стилизации",
            reply_markup=keyboard
        )

        await state.clear()

    except Exception as e:
        logger.exception(f"AI style error: {str(e)}")
        # await message.reply(f"❌ Ошибка: {str(e)}")
        # await state.clear()
```

ПРИЛОЖЕНИЕ 2

ЛИСТИНГ ФАЙЛА КОНФИГУРАЦИИ СЕРВИСА

```
config.py

from aiogram import Bot, Dispatcher
import logging
import os
from datetime import datetime

# Создаем папку для логов
os.makedirs("logs", exist_ok=True)

# Генерируем имя файла с датой и временем
log_filename = datetime.now().strftime("logs/bot_%Y-%m-%d_%H-%M-%S.log")

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler("bot.log"),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)

API_TOKEN = '7211758777:AAG1FVB3x6lwdFPAXCaG3aT_M93YGB_s1dA'
bot = Bot(token=API_TOKEN)
dp = Dispatcher()

user_images = {}
```

ПРИЛОЖЕНИЕ 3

ЛИСТИНГ ОСНОВНОГО ФАЙЛА СЕРВИСА

```
import asyncio
from config import dp, bot, logger
from aiogram import Bot, Dispatcher
import handlers

async def main():
    logger.info("Starting bot")
    try:
        await dp.start_polling(bot, skip_updates=True)
    except Exception as e:
        logger.critical(f"Bot crashed: {str(e)}")
    finally:
        logger.info("Bot stopped")
if __name__ == '__main__':
    print("Бот запущен!")
    asyncio.run(main())
```

ПРИЛОЖЕНИЕ 4

ДИАГРАММА СЦЕНАРИЕВ ВЗАИМОДЕЙСТВИЯ С БОТОМ

